

FoliaScan-Azure

✓ Phase 1 — Build and validate the model locally

Goal

The goal of Phase 1 is to build and verify a complete local machine-learning workflow before introducing Azure infrastructure.

Working locally first reduces cost and complexity and makes problems easier to debug. By the end of this phase, the data preparation, model pipeline, training process, checkpoint selection, and final evaluation should work reliably and reproducibly.

Complete workflow

```
Set up a reproducible Python project
↓
Prepare and validate the dataset
↓
Create leakage-safe data splits
↓
Load data through a manifest
↓
Build the PyTorch data and model pipeline
↓
Train and validate a baseline model
↓
Select the best checkpoint
↓
Evaluate it on the untouched test set
↓
Analyse model errors
```

Main concepts

1. Data leakage

Data leakage occurs when information from validation or test data indirectly influences model training, making evaluation results unrealistically good.

In FoliaScan, related images from the same physical leaf must remain in the same split:

```
One leaf_id → one final split
```

The general lesson is that data should be split using the true independent unit, such as a patient, customer, device, video, or physical object.

2. Train, validation, and test splits

Each split has a different role:

- **Train:** updates model parameters.
- **Validation:** compares epochs and selects the best model.
- **Test:** evaluates the final selected model once.

The test set must not influence hyperparameters, early stopping, checkpoint selection, or model architecture.

3. Manifest-driven datasets

A manifest is a structured file that describes the dataset, including information such as:

```
relative_path
class_name
split
leaf_id
source_split
```

The images remain unchanged, while the manifest determines how they are used.

This makes the workflow easier to audit, reproduce, version, and move between local and cloud environments.

4. Reproducibility

A reproducible workflow records enough information to recreate and compare experiments.

Important elements include:

- Version-controlled code
- Locked dependency versions
- Fixed random seeds
- Deterministic class mappings and splits
- Saved configurations
- Recorded metrics
- Saved checkpoints

Reproducibility does not always mean completely identical GPU results, but uncontrolled differences should be minimised and important settings should be recorded.

5. End-to-end validation

Individual functions may work correctly while the complete system still fails.

Phase 1 therefore validates the full connection:

data → transforms → batches → model → training → checkpoint → evaluation

Key takeaway

The purpose of Phase 1 is not only to train a classifier. It is to create a trustworthy local ML workflow that can later be moved to Azure without changing its fundamental data, training, and evaluation logic.

- 1.1 Project foundation

Objective

Create a structured and reproducible Python project in which dependencies, source code, tests, configurations, and development tools are organised before implementing the ML pipeline.

What was implemented

FoliaScan was created as a Python 3.11 project using:

- Poetry for project and dependency management
- A src-based package layout
- pytest for testing
- Ruff for linting
- mypy for static type checking
- Feature branches and pull requests for controlled development

These are the actual foundations described for subphase 1.1 in the original notes.

Concepts I should understand

1. Dependency management

A project depends on external packages such as PyTorch, Pillow, pandas, and Azure libraries.

Dependency management records:

- Which packages the project needs
- Which versions are compatible
- Which packages are required only for development

Dependencies are commonly divided into:

- **Runtime dependencies:** required to run the application
- **Development dependencies:** required for testing, linting, typing, or development

2. Poetry and virtual environments

Poetry manages the Python project and creates an isolated virtual environment.

The virtual environment prevents FoliaScan's dependencies from conflicting with packages used by other projects.

```
poetry install
```

```
poetry run <command>
```

- poetry install installs the project dependencies.
- poetry run executes a command inside FoliaScan's environment.

3. pyproject.toml and poetry.lock

pyproject.toml describes the project and its dependency requirements.

It may include:

- Project metadata
- Python version
- Runtime dependencies
- Development dependencies
- Tool configurations

poetry.lock records the exact package versions selected by Poetry.

pyproject.toml → allowed dependency versions

poetry.lock → exact resolved versions

The lock file helps local machines, CI/CD systems, and deployment environments install consistent dependencies.

4. The src layout

The project package is stored under:

```
src/  
└─ foliascan/
```

A simplified structure is:

```
foliascan-azure/  
├─ src/foliascan/  
├─ tests/  
├─ configs/  
├─ docs/  
├─ data/  
├─ infra/  
├─ pyproject.toml  
└─ poetry.lock
```

The src layout separates application code from project configuration and reduces the risk of importing code accidentally from the repository root instead of from the installed package.

5. Testing, linting, and type checking

These tools detect different categories of problems:

Tool | Main responsibility

pytest | Checks program behaviour

Ruff | Detects code-quality and style problems

mypy | Checks type consistency

Examples:

- pytest can verify that a splitting function keeps one leaf in one split.
- Ruff can find unused imports or common coding problems.
- mypy can detect incorrect value types before runtime.

Passing one tool does not replace the others.

6. Feature branches and pull requests

A feature branch isolates one logical change from the stable main branch.

```
main
└─ feat/new-feature
```

This makes changes easier to:

- Review
- Test
- Reverse
- Merge safely
- Validate later with CI/CD

Development should normally happen on a feature branch rather than directly on main.

Important note

Never commit:

- Passwords or API keys
- Azure credentials
- Real .env files
- Raw datasets
- Large generated artifacts
- Temporary training outputs

These files may contain sensitive information, consume unnecessary repository space, or be generated again from code.

Essential commands

Install the environment

```
poetry install
```

Run quality checks

```
poetry run pytest  
poetry run ruff check .  
poetry run mypy src
```

Start a feature branch

```
git switch main  
git pull --ff-only origin main  
git switch -c feat/descriptive-name
```

Review and commit changes

```
git status  
git diff  
git add <files>  
git diff --staged  
git commit -m "feat: describe the change"
```

Review questions

1. What is the difference between runtime and development dependencies?
2. What different information is stored in pyproject.toml and poetry.lock?
3. Why is an isolated virtual environment useful?
4. What different problems do pytest, Ruff, and mypy detect?
5. Why is a src-based layout useful?
6. Why should development happen on feature branches?
7. Why must credentials and generated artifacts be excluded from Git?

Key takeaway

A professional ML project needs more than model code. Dependency management, isolated environments, clear source organisation, automated checks, and version-control practices make the project reproducible and maintainable.

- 1.2 Generic dataset preparation

Objective

Create a reusable pipeline for discovering, validating, describing, and splitting folder-based image datasets before model training.

What was implemented

The generic dataset tools can:

- Discover class folders and image files.
- Validate that images can be opened.
- Report corrupted or unsupported files.
- Summarise image formats, dimensions, and colour modes.
- Count the number of images in each class.
- Create deterministic train, validation, and test assignments.
- Save the results as structured metadata.

Concepts I should understand

1. Raw data and processed metadata

Raw data contains the original image files:

`data/raw/`

Processed metadata describes those files:

`data/processed/dataset_manifest.csv`
`data/processed/dataset_report.json`

The raw images remain unchanged. Reports and manifests record how the dataset was validated and organised.

2. Image validation

An image file may exist but still be unusable because it is:

- Corrupted
- Empty
- Unsupported
- Incorrectly encoded
- Impossible to decode

Dataset validation detects these problems before training starts.

A long training job should not fail because one invalid image was discovered too late.

3. Dataset inspection

Inspection helps answer questions such as:

- How many classes exist?
- How many images belong to each class?
- Are some classes much larger than others?
- Are image formats and dimensions consistent?
- Are any files corrupted?

Inspection does not modify the dataset. It creates information needed to make better training decisions.

4. Class imbalance

Class imbalance occurs when some classes contain many more examples than others.

This matters because a model may achieve good overall accuracy by performing well on large classes while ignoring smaller classes.

At this stage, the important task is to detect and report imbalance. Possible solutions can be considered later during training and evaluation.

5. Deterministic dataset preparation

A fixed random seed makes the same split reproducible:

```
random_seed = 42
```

The same data, code, and seed should generate the same assignments. This makes model comparisons more reliable.

Important note

The generic folder-based splitter works at the individual-image level. It is suitable only when images can be treated as independent samples.

It is not used for PlantVillage because PlantVillage contains related images identified by leaf_id.

Essential commands

```
# Inspect a folder-based dataset
poetry run python -m foliascan.data.cli inspect `
  --data-dir data/raw/plantvillage_tomato_color` # Create a generic deterministic mani
poetry run python -m foliascan.data.cli split `
  --data-dir data/raw/plantvillage_tomato_color`
  --output data/processed/dataset_manifest.csv
```

Review questions

1. Why should images be validated before training?
2. What is the difference between raw data and processed metadata?
3. How can class imbalance make accuracy misleading?
4. Why should dataset preparation be reproducible?
5. When is an individual-image split unsafe?

Key takeaway

Dataset preparation should be an automated and repeatable pipeline, not a manual one-time task. Validating the data early prevents silent errors from affecting training and evaluation.

- 1.3 PlantVillage ingestion & safe splitting

Objective

Import the official PlantVillage Tomato dataset and create leakage-safe train, validation, and test assignments based on physical leaf groups.

What was implemented

The PlantVillage workflow:

- Loads mohanty/PlantVillage from Hugging Face.
- Uses the colour-image configuration.
- Filters the dataset to Tomato images.
- Exports ten tomato-condition classes.
- Creates a source manifest.
- Preserves the official test records.
- Groups related images using leaf_id.
- Creates the final FoliaScan manifest.
- Resolves leaf groups appearing across source splits.

Concepts I should understand

1. Dataset-specific ingestion

Generic tools cannot always handle the structure of an external dataset correctly.

PlantVillage contains useful metadata such as:

```
crop
disease
leaf_id
source_split
```

The ingestion pipeline uses this metadata instead of treating the dataset as a simple collection of unrelated images.

2. Source manifest

The source manifest records the original information obtained from PlantVillage:

```
relative_path
class_name
source_split
leaf_id
```

It preserves where each image came from before FoliaScan applies its own final split rules.

3. Final FoliaScan manifest

The final manifest adds:

`split`

The difference is:

- `source_split`: the original PlantVillage assignment
- `split`: the final leakage-safe FoliaScan assignment

Keeping both columns makes the transformation auditable.

4. Group-aware splitting

The splitting unit is the physical leaf, not the individual image.

```
leaf_id = 500
|-- image_1.jpg
|-- image_2.jpg
|-- image_3.jpg
```

All images belonging to this `leaf_id` must receive the same final split.

This prevents the model from seeing one view of a leaf during training and another view of the same leaf during evaluation.

5. Test precedence

Some related leaf records appeared across the official training and test splits.

FoliaScan applies this rule:

When a `leaf_id` appears in the official test set, every record with that `leaf_id` is assigned to the final test set.

The original `source_split` remains stored, so the change can still be inspected.

6. Integration testing of external datasets

Documentation describes how a dataset is expected to behave. The real records may contain unexpected cases.

External datasets should therefore be checked for:

- Missing metadata
- Duplicate identifiers
- Unexpected classes
- Invalid files
- Groups appearing across multiple splits

Important note

The generic FoliaScan split command must not be used for PlantVillage because it does not understand leaf_id relationships.

Essential commands

```
# Export the official Tomato subset
poetry run python -m foliascan.data.cli plantvillage-export `
  --output-dir data/raw/plantvillage_tomato_color `
  --source-manifest data/processed/plantvillage_source_manifest.csv# Create the leak
poetry run python -m foliascan.data.cli plantvillage-split `
  --source-manifest data/processed/plantvillage_source_manifest.csv `
  --output data/processed/dataset_manifest.csv `
  --validation-ratio 0.15 `
  --random-seed 42
```

Review questions

1. What does leaf_id represent?
2. Why is the generic image-level split unsafe for PlantVillage?
3. What is the difference between source_split and split?
4. Why is the official test split preserved?
5. Why does FoliaScan use test precedence?

Key takeaway

Splitting logic must reflect how the data was generated. For PlantVillage, the physical leaf is the independent unit, so all related images must remain together.

- 1.4 PyTorch data pipeline & smoke test

Objective

Connect the manifest, image loading, transformations, batching, and ResNet18 model into one working pipeline before full training.

What was implemented

The pipeline includes:

```
Manifest
↓
PyTorch Dataset
↓
Image transforms
↓
DataLoader
↓
ResNet18
↓
Class scores
```

It supports:

- Image loading and RGB conversion
- Deterministic class mapping
- Training augmentation
- Validation and test transforms
- ImageNet normalisation
- Train, validation, and test DataLoaders
- A ResNet18 classifier with ten outputs
- A forward-pass smoke test

Concepts I should understand

1. PyTorch Dataset

A Dataset defines how one sample is loaded.

```
Read one manifest row
↓
Locate the image
↓
Open and convert to RGB
↓
Apply transforms
↓
Return image tensor and class index
```

The Dataset answers:

How do I load one example?

2. PyTorch DataLoader

A DataLoader retrieves Dataset samples and combines them into batches.

32 individual samples → one batch

It also manages:

- Batch size
- Shuffling
- Batch order
- Parallel loading

The DataLoader answers:

How do I efficiently provide samples to the model?

3. Image transforms

Transforms prepare images for the model.

Training transforms include small random changes such as:

- Horizontal flipping
- Small rotations
- Conservative crop variation

Validation and test transforms are deterministic:

- Resize
- Centre crop
- Convert to tensor
- Normalise

Random augmentation is used only during training. Evaluation should not change randomly between runs.

4. ImageNet normalisation

ResNet18 was pretrained on ImageNet using a specific input normalisation.

Using the corresponding normalisation makes FoliaScan images more compatible with the pretrained model.

5. Class mapping

The model uses integer labels rather than class names:

```
Tomato___Bacterial_spot → 0
Tomato___Early_blight   → 1
...
Tomato___healthy         → 9
```

The same mapping must be used during training, validation, testing, and inference.

It is saved in the checkpoint so predictions can later be converted back to class names.

6. Tensor shapes

A typical image batch has the shape:

```
[32, 3, 224, 224]
```

This means:

```
32  images
3   RGB channels
224 image height
224 image width
```

For ten classes, the model output is:

```
[32, 10]
```

Each image receives ten class scores.

7. Transfer learning

ResNet18 has already learned useful general visual features from ImageNet.

FoliaScan replaces the original 1,000-class output layer with a ten-class layer:

```
Pretrained feature extractor
      ↓
New FoliaScan classification layer
```

This provides a practical baseline without training an entire neural network from random initialisation.

8. Smoke test

A smoke test is a small end-to-end integration check.

It verifies that:

- Manifest paths are correct
- Images can be loaded
- Transforms work
- Labels are valid
- A batch can be created
- The model accepts the batch
- Output dimensions are correct

It does not update model parameters.

Important note

A unit test checks a small component in isolation. A smoke test checks whether several real components work correctly together.

Essential command

```
poetry run python -m foliascan.training.smoke_test `
  --manifest data/processed/dataset_manifest.csv `
  --data-dir data/raw/plantvillage_tomato_color `
  --config configs/training.example.yaml `
  --split train `
  --device cuda
```

Review questions

1. What is the difference between a Dataset and a DataLoader?
2. What does [32, 3, 224, 224] represent?
3. Why must all splits share the same class mapping?
4. Why is random augmentation used only for training?
5. What does a smoke test verify?
6. Why was the final ResNet18 layer replaced?

Key takeaway

Before spending time on training, verify that the complete path from manifest to model

output works correctly.

- 1.5 Local baseline training

Objective

Turn the working data and model pipeline into a reproducible training workflow that produces checkpoints and training history.

What was implemented

The training workflow includes:

- CPU or CUDA device selection
- Fixed random seeds
- Training and validation loops
- Cross-entropy loss
- AdamW optimisation
- Best and last checkpoints
- Early stopping
- CSV and JSON training history
- Output-directory protection
- A command-line training interface

The test split is not used during training or checkpoint selection.

Concepts I should understand

1. Forward pass

The image batch passes through the model:

`image batch` → `ResNet18` → `class scores`

The raw output scores are called logits.

2. Cross-entropy loss

`CrossEntropyLoss` compares the model's class scores with the correct class labels.

It is suitable because each image belongs to one of ten mutually exclusive classes.

The loss gives the training process a numerical measure of prediction error.

3. Backpropagation

Backpropagation calculates gradients for trainable model parameters.

Gradients indicate how the parameters should change to reduce the loss.

4. AdamW optimiser

AdamW uses the gradients to update model parameters.

It also applies weight decay, which can help limit overfitting by discouraging unnecessarily large parameter values.

5. Training and validation loops

During training:

- Gradients are calculated.
- Parameters are updated.
- Random augmentation may be used.

During validation:

- Gradients are disabled.
- Parameters remain unchanged.
- Deterministic transforms are used.
- Generalisation is measured.

6. Epoch

One epoch means the model has processed the complete training dataset once.

After each epoch, validation is performed and the result is compared with earlier epochs.

7. Checkpointing

FoliaScan saves:

```
best_model.pt
last_model.pt
```

- `best_model.pt` stores the epoch with the lowest validation loss.
- `last_model.pt` stores the most recently completed epoch.

The final epoch is not necessarily the best epoch.

A checkpoint also records information such as:

- Model state
- Optimiser state
- Class mapping
- Training configuration
- Random seed
- Epoch and metrics

8. Early stopping

Early stopping ends training when validation loss does not improve for a configured number of epochs.

It saves compute and prevents training from continuing without useful improvement.

9. Training history

The training process writes:

`history.csv`
`history.json`

These files record training and validation metrics for each epoch.

They can later support learning curves, experiment comparison, and MLflow tracking.

10. Baseline model

A baseline is a dependable reference model, not necessarily the best possible model.

It helps answer:

- Does training work end to end?
- What can a simple model achieve?
- Do later changes improve the result?
- Can the same workflow run locally and in Azure?

Important note

Validation selects the checkpoint. The test set must remain untouched until model development is complete.

Results from a one-epoch integration check prove that the pipeline works; they should not be reported as final model performance.

Essential command

```
poetry run python -m foliascan.training.train `
  --manifest data/processed/dataset_manifest.csv `
  --data-dir data/raw/plantvillage_tomato_color `
  --config configs/training.example.yaml `
  --epochs 10 `
  --device cuda `
  --output-dir artifacts/training/resnet18_baseline_v1
```

Review questions

1. What happens during the forward pass?
2. What is the difference between backpropagation and the optimiser?
3. Why are gradients disabled during validation?
4. Why can best_model.pt be better than last_model.pt?
5. What problem does early stopping address?
6. Why is a baseline needed before trying more complex models?

Key takeaway

A reliable training workflow must manage loss, gradients, optimisation, validation, checkpointing, outputs, and reproducibility—not only run the model for several epochs.

- 1.6 Evaluation & error analysis

Objective

Evaluate the selected checkpoint on the untouched test split and investigate the model's main failure patterns.

What was implemented

The evaluator:

- Loads best_model.pt.
- Restores the model configuration and class mapping.
- Loads only test records.
- Uses deterministic transforms.
- Disables gradient calculation.
- Calculates aggregate and per-class metrics.
- Creates a confusion matrix.
- Saves predictions and misclassification reports.

Concepts I should understand

1. Evaluation mode

During evaluation:

```
model.eval()
```

switches layers such as dropout and batch normalisation to evaluation behaviour.

```
torch.inference_mode()
```

disables gradient tracking because model parameters are not being updated.

This reduces memory use and unnecessary computation.

2. Accuracy

Accuracy is the proportion of all predictions that are correct.

It is useful as a general summary but does not show which classes are difficult.

3. Precision

Precision asks:

When the model predicted this class, how often was it correct?

High precision means few false positives.

4. Recall

Recall asks:

Of all real examples of this class, how many did the model find?

High recall means few missed examples.

5. F1-score

F1 combines precision and recall.

It is useful when both false positives and false negatives matter.

6. Macro and weighted metrics

Macro metrics give every class equal importance.

Weighted metrics give more influence to classes containing more samples.

Macro F1 is especially useful when class sizes are unequal and performance on smaller classes matters.

7. Confusion matrix

A confusion matrix shows which classes are confused with each other.

For example:

True class: Early blight
Predicted class: Late blight

Repeated confusion between the same classes may indicate visual similarity or insufficient model generalisation.

8. Misclassification analysis

The evaluator records information such as:

- Image path
- True class
- Predicted class
- Confidence
- Correctness

High-confidence wrong predictions are particularly useful for finding:

- Incorrect labels
- Similar classes
- Background shortcuts
- Unusual images
- Weak generalisation

9. Domain shift

PlantVillage images were collected under relatively controlled conditions.

Real field images may have:

- Complex backgrounds
- Different lighting
- Occluded leaves
- Different cameras
- Weather effects
- Multiple diseases

Strong test results on PlantVillage therefore do not automatically guarantee strong real-world performance.

Important note

The test results must not be used repeatedly to tune the model. Once test results influence model changes, the test set is no longer an independent final evaluation.

Essential command

```
poetry run python -m foliascan.evaluation.evaluate `
--manifest data/processed/dataset_manifest.csv `
--data-dir data/raw/plantvillage_tomato_color `
--checkpoint artifacts/training/resnet18_baseline_v1/best_model.pt `
--output-dir artifacts/evaluation/resnet18_baseline_v1 `
--device cuda
```

Review questions

1. Why are gradients disabled during evaluation?
2. What is the difference between precision and recall?
3. Why can macro F1 be more informative than accuracy?
4. What information does a confusion matrix provide?
5. Why should high-confidence wrong predictions be inspected?
6. Why does strong PlantVillage performance not guarantee field performance?

Key takeaway

Evaluation should explain not only how often the model is correct, but also which classes fail, how they fail, and whether the test data represents the intended real-world environment.

✓ Phase 2 — Create Azure infrastructure

Goal

The goal of Phase 2 is to create a secure, reproducible, and cost-conscious Azure Machine Learning environment for FoliaScan.

The local ML workflow already exists. This phase prepares the cloud resources required to store assets, run jobs, manage models, and support later deployment.

Complete workflow

Verify the Azure account and subscription



Create and organise the resource group



Create the Azure ML workspace



Provision CPU and GPU compute



Review identities, permissions, and costs



Connect the local project to Azure

Main concepts

1. Azure resource hierarchy

Azure resources are organised through this hierarchy:

Azure account



Subscription



Resource group



Individual resources

- A **subscription** is the billing, quota, and access boundary.
- A **resource group** organises related resources.
- Resources include the Azure ML workspace, storage, compute, and supporting services.

2. Azure Machine Learning workspace

The workspace is the central management area for an Azure ML project.

It connects:

- Data assets
- Environments
- Jobs and experiments
- Compute resources
- Models
- Endpoints
- Logs and monitoring

The workspace manages these components but is not itself the machine that trains the model.

3. Reproducible infrastructure

Cloud resources should be created from reviewable configurations and commands rather than only through manual portal clicks.

FoliaScan uses:

- Azure CLI
- Azure ML CLI v2
- YAML configuration files

This makes infrastructure easier to reproduce, inspect, and later automate.

4. Compute and cost control

Cloud compute is paid infrastructure. The project therefore separates CPU and GPU workloads and uses scale-to-zero settings to avoid paying for inactive machines.

5. Identity and access

Cloud resources must know:

- Who or what is connecting
- How that identity authenticates
- What it is authorised to do
- Where the permission applies

Credentials should not be stored directly in source code.

Key takeaway

Phase 2 creates the managed cloud foundation around the local ML workflow. A correct Azure setup requires more than creating a VM: subscription, region, workspace, compute, identity,

permissions, and cost controls must work together.

- 2.1 Azure readiness and subscription setup

Objective

Verify that the local computer and Azure account are ready before creating cloud resources.

What was implemented

The setup confirmed that:

- Azure CLI was installed.
- The account could authenticate.
- The correct subscription was active.
- The Azure ML CLI extension was available.
- Required resource providers were registered.
- Azure ML-supported regions could be inspected.

Concepts I should understand

1. Azure subscription

A subscription controls:

- Billing
- Available quotas
- Regions and services
- User permissions
- Spending limits

Azure CLI performs operations in one active subscription at a time.

Selecting the wrong subscription can cause unexpected billing, missing permissions, or resources being created in the wrong environment.

2. Azure CLI and Azure ML extension

Azure CLI is the general command-line interface:

`az`

The Azure ML extension adds machine-learning commands:

`az ml`

The relationship is:

```
Azure CLI
└─ Azure ML extension
    └─ workspace, compute, data, job, model and endpoint commands
```

3. Resource provider

A resource provider enables Azure to create and manage a particular resource type.

Examples:

```
Microsoft.MachineLearningServices → Azure Machine Learning
Microsoft.Storage                  → Storage accounts
Microsoft.KeyVault                  → Key Vault
```

A provider may need to be registered before its resources can be created.

4. Azure region

A region is the geographical location where Azure resources are hosted.

Region choice affects:

- Service availability
- VM and GPU availability
- Quota
- Cost
- Latency
- Data-location requirements

The nearest region is not automatically the best region for every workload.

Important note

Always verify the active subscription before creating paid resources. Subscription IDs, tenant IDs, and credentials should not appear in public screenshots or documentation.

Essential commands

```
az version
az login
az account show --output table
az extension add --name ml
az ml --help
az provider show --namespace Microsoft.MachineLearningServices --query "{Provider:namespace, State:registrationState}" --output table
```


Review questions

1. What is the role of an Azure subscription?
2. What could happen if the wrong subscription is active?
3. What is the relationship between az and az ml?
4. Why must resource-provider registration be checked?
5. Why can region choice affect an ML workload?

Key takeaway

Before creating infrastructure, confirm the subscription, authentication, CLI tools, providers, permissions, and region. Many Azure failures come from configuration rather than ML code.

- 2.2 Resource naming and resource group

Objective

Create a dedicated resource group and establish a consistent naming convention for FoliaScan resources.

What was implemented

The following decisions were made:

Region: Norway East
Resource group: rg-foliascan-dev-ne
Workspace: mlw-foliascan-dev-ne
Environment: Development

Naming pattern:

<resource-type>-<project>-<environment>-<region>

Example:

rg-foliascan-dev-ne

Concepts I should understand

1. Resource group

A resource group is a logical container for related Azure resources.

For FoliaScan, it can contain:

rg-foliascan-dev-ne
├─ Azure ML workspace
├─ Storage account
├─ Key Vault
├─ Container Registry
└─ Compute resources

It supports organisation, access control, cost inspection, and cleanup.

2. Naming convention

A consistent name should make the resource's purpose understandable.

Useful parts include:

- Resource type
- Project
- Environment
- Region

Clear names become especially important when a company manages many projects and environments.

3. Environment separation

Resources from different lifecycle stages should be separated:

```
dev → development
test → testing
prod → production
```

Development changes should not accidentally affect production resources or costs.

4. Tags

Tags are key-value metadata attached to resources.

Examples:

```
project=foliascan
environment=dev
purpose=learning
```

They help with:

- Cost tracking
- Filtering
- Ownership
- Governance

Important note

Deleting a resource group may delete all resources inside it. Always inspect its contents first.

Essential commands

```
az group create `
  --name rg-foliascan-dev-ne `
  --location norwayeast `
```

```
--tags project=foliascan environment=dev purpose=learningaz group show `
--name rg-foliascan-dev-ne `
--output tableaz resource list `
--resource-group rg-foliascan-dev-ne `
--output table
```

Review questions

1. What is the purpose of a resource group?
2. Why should development and production resources be separated?
3. How does a naming convention improve cloud management?
4. Why are tags useful?
5. What is the risk of deleting a resource group?

Key takeaway

Cloud resources should be organised before they are created. Resource groups, clear names, environments, and tags make the infrastructure easier to understand, manage, and clean up.

- **2.3 Azure Machine Learning workspace**

Objective

Create the central Azure Machine Learning workspace using a version-controlled YAML configuration.

What was implemented

A development workspace was created:

Workspace: mlw-foliascan-dev-ne
Resource group: rg-foliascan-dev-ne
Region: Norway East

It was defined in:

infra/azure/workspace.yml

Concepts I should understand

1. Azure ML workspace

The workspace manages and connects:

- Data assets
- Environments
- Compute
- Jobs and experiments
- Models
- Endpoints
- Logs

It is a management hub, not a training machine.

Training happens on a compute target connected to the workspace.

2. Supporting resources

An Azure ML workspace uses supporting Azure services.

Storage account

Stores data, job outputs, model artifacts, and logs.

Key Vault

Stores secrets and sensitive configuration securely.

Container Registry

Stores container images used for training and deployment environments.

Application Insights

Collects monitoring, request, performance, and error information.

These services have different responsibilities even though the workspace connects them together.

3. Declarative YAML configuration

A YAML file describes the desired workspace configuration.

Benefits compared with portal-only creation:

- Version controlled
- Reviewable
- Repeatable
- Easier to automate
- Less dependent on manual steps

4. Provisioning state

Provisioning is the process of creating or updating a cloud resource.

A state such as:

Succeeded

means Azure completed the requested operation.

A failed state requires inspection of the region, permissions, configuration, quota, or service availability.

Important note

Creating the workspace may also create supporting resources. Some of them can produce small costs even when no GPU job is running.

Essential commands

```
az ml workspace create `
  --file infra/azure/workspace.ymlaz ml workspace show `
  --name mlw-foliascan-dev-neaz configure --defaults `
  group=rg-foliascan-dev-ne `
  workspace=mlw-foliascan-dev-ne `
  location=norwayeast
```

Review questions

1. Why is the workspace not the same as compute?
2. What does the workspace manage?
3. What roles do Storage, Key Vault, Container Registry, and Application Insights play?
4. Why is YAML preferable to portal-only configuration?
5. What does provisioning state describe?

Key takeaway

The workspace is the central control and management layer of Azure Machine Learning. It connects data, software environments, compute, jobs, models, and deployment resources.

- 2.4 CPU and GPU compute

Objective

Create managed compute resources for preprocessing, testing, and model-training jobs.

What was implemented

A managed CPU cluster was created:

Name:	cpu-fs-dev
VM size:	Standard_DS3_v2
Minimum nodes:	0
Maximum nodes:	1
Idle scale-down:	120 seconds
Tier:	Dedicated

GPU availability and quota were investigated separately. The notes record an initial regional VM-size failure, while the later connection test confirms that both cpu-fs-dev and gpu-fs-dev were discovered with a Succeeded state.

Concepts I should understand

1. Compute target

A compute target is the resource on which Azure ML executes work.

Examples include:

- Compute cluster
- Compute instance
- Kubernetes
- Other connected platforms

2. Compute cluster versus compute instance

Compute instance	Compute cluster
Interactive development machine	Job-oriented compute

Often used by one person	Used for submitted workloads
--------------------------	------------------------------

Suitable for notebooks and debugging	Suitable for repeatable training
--------------------------------------	----------------------------------

May stay running	Can scale down automatically
------------------	------------------------------

FoliaScan uses clusters because its main purpose is running submitted jobs rather than maintaining a personal cloud workstation.

3. CPU and GPU workloads

CPU compute is suitable for:

- Validation
- Data preparation
- Small smoke tests
- Lightweight scripts

GPU compute is suitable for:

- Neural-network training
- Large tensor operations
- Compute-intensive deep learning

Using separate clusters avoids paying GPU prices for tasks that do not need a GPU.

4. VM size and VM family

A VM size defines the hardware of one node:

- CPU cores
- Memory
- GPU type and count
- GPU memory

Related VM sizes belong to a VM family.

Azure quotas are often assigned to the family rather than only to one VM size.

5. Availability and quota

These are separate checks:

- **Availability:** Does Azure offer this VM size in the region?
- **Quota:** Is this subscription allowed to use enough cores from its family?

A VM can appear in a list but still fail because of:

- Zero family quota
- Insufficient available quota
- Unsupported region
- Temporary Azure capacity limits

6. Autoscaling and scale to zero

FoliaScan uses:

`min_instances: 0`

`max_instances: 1`

- Minimum zero prevents paying for idle nodes.
- Maximum one avoids accidental multi-node costs and stays within quota.

7. Dedicated and low-priority compute

- **Dedicated compute:** more reliable, but more expensive.
- **Low-priority compute:** cheaper spare capacity that Azure may interrupt.

Low-priority machines are more suitable for retryable experiments than for the first complete cloud workflow.

Important note

Disabling public SSH access and disabling public IP addresses are different security settings. No public SSH prevents direct login, but the node may still have a public IP unless that is disabled separately.

Essential commands

```
az ml compute list
az ml compute list-sizes --location norwayeast
az ml compute list-usage --location norwayeast
az ml compute create --file infra/azure/compute/cpu-cluster.yml
```

Review questions

1. What is the difference between a compute cluster and a compute instance?
2. Why should CPU and GPU tasks use separate compute?
3. What is the difference between VM size and VM family?
4. Why can VM creation fail even when the size appears available?
5. Why is `min_instances: 0` useful?
6. When is low-priority compute appropriate?
7. What caused the original GPU provisioning problem?

Key takeaway

Compute selection requires balancing workload requirements, regional availability, quota, reliability, security, and cost—not simply choosing the most powerful machine.

- 2.5 Identity, permissions, and cost controls

Objective

Understand who can access the Azure resources, what they are allowed to do, and how unnecessary spending is reduced.

What was implemented

- User access and inherited roles were inspected.
- The learning subscription's Owner access was identified.
- The workspace identity was reviewed.
- No unnecessary role assignments were added.
- Both clusters were configured to scale from zero to one node.
- Paid jobs were avoided without confirmed free credits.

Concepts I should understand

1. Identity

An identity can be:

- A user
- An application
- An Azure resource

It represents who or what is requesting access.

2. Authentication and authorization

- **Authentication:** verifies who the identity is.
- **Authorization:** determines what that identity may do.

Successful login does not automatically mean permission to perform every operation.

3. Azure RBAC

Azure Role-Based Access Control combines:

`identity + role + scope`

Example:

`User + Contributor role + FoliaScan resource group`

4. Scope

Scope defines where the permission applies:

Subscription
↓
Resource group
↓
Individual resource

A smaller scope is safer when wider access is unnecessary.

5. Managed identity

A managed identity allows an Azure resource to authenticate to another Azure service without storing a password or secret in the source code.

6. Least privilege

An identity should receive only the permissions it needs.

Owner access is practical for a personal learning subscription but is normally too broad for an employee working in a company environment.

7. Cost controls

Important controls include:

- Scale to zero
- Maximum node limits
- Short idle time
- Choosing suitable VM sizes
- Checking active resources
- Budget alerts

A budget alert sends a warning. It does not automatically stop resources or guarantee zero cost.

Important note

Autoscaling controls compute-node cost, but the workspace and supporting services may still produce charges. Scale-to-zero does not mean the complete Azure environment is free.

Essential commands

```
az ad signed-in-user show
az role assignment list `
  --assignee <object-id>
az ml workspace show --query identity
az ml compute list
```

Review questions

1. What is the difference between authentication and authorization?
2. What three elements form an RBAC assignment?
3. Why are managed identities safer than credentials in code?
4. What does permission scope mean?
5. Why is least privilege important?
6. Why does scale-to-zero not guarantee zero Azure cost?
7. Why is a budget alert not a hard spending limit?

Key takeaway

Responsible cloud engineering requires secure identities, limited permissions, appropriate scopes, and active cost controls. Access and spending should never be left to default assumptions.

- 2.6 Local-to-Azure connection test

Objective

Confirm that the local FoliaScan Python environment can authenticate and connect to the real Azure ML workspace without submitting jobs or changing resources.

What was implemented

A read-only connection test was implemented using Azure ML SDK v2.

It confirmed:

Connection: connected
Workspace: mlw-foliascan-dev-ne
Region: Norway East

cpu-fs-dev → AmlCompute → Succeeded
gpu-fs-dev → AmlCompute → Succeeded

The test did not:

- Upload data
- Start compute
- Submit training
- Create resources
- Modify existing resources

Concepts I should understand

1. Azure ML SDK v2

The SDK allows Python code to work programmatically with:

- Workspaces
- Compute
- Data
- Environments
- Jobs
- Models
- Endpoints

Azure CLI is useful for shell commands and automation. The SDK is useful when Azure operations are part of a Python application.

2. MLClient

MLClient is the main Python client for connecting to a specific Azure ML workspace.

It needs the:

- Subscription
- Resource group
- Workspace name
- Authentication credential

After connecting, it can inspect or manage Azure ML resources.

3. DefaultAzureCredential

DefaultAzureCredential tries supported authentication methods without hardcoding credentials.

During local development, it can reuse the existing Azure CLI login created with:

```
az login
```

4. Environment variables

Values such as the subscription ID, resource group, and workspace name should be kept outside the application code.

Example variable names:

```
AZURE_SUBSCRIPTION_ID
AZURE_RESOURCE_GROUP
AZURE_ML_WORKSPACE
```

Real secret values should not be committed.

5. Read-only operation

A read-only test retrieves existing information without creating, updating, or deleting resources.

Listing compute proves that authentication, permissions, workspace configuration, and

network access work together.

6. Real integration test versus mock

A mocked test checks application logic without contacting Azure.

A real read-only test confirms that the actual Azure account and workspace can be reached.

Both are useful:

- Mocks support fast automated tests.
- A real read confirms actual cloud connectivity.

Important note

Discovering a compute cluster does not start it. Submitting a job is a separate action that may activate nodes and generate costs.

Essential commands

```
az loginpoetry run python -m foliascan.cloud.azure_connection
```

Review questions

1. What is the role of MLClient?
2. How can DefaultAzureCredential authenticate locally?
3. Why should Azure identifiers and credentials not be hardcoded?
4. Why is a real workspace read useful?
5. What is the difference between listing compute and submitting a job?
6. Why are mocked integration tests still necessary?

Key takeaway

The connection test proves that the local project can securely reach and inspect the real Azure ML environment while keeping configuration outside the code and avoiding paid or destructive operations.

✓ Phase 3 — Move and Version the Dataset in Azure

Goal

The goal of Phase 3 is to move the verified local dataset into Azure and register it as reusable, versioned Azure Machine Learning data assets.

This allows future training jobs to reference stable asset names and versions instead of depending on local folders or manually entered storage paths.

Complete workflow

```
Inspect the workspace storage and datastores
↓
Define the required Azure data assets
↓
Describe the assets in version-controlled YAML
↓
Upload and register the assets
↓
Verify their metadata and local data integrity
```

Main concepts

1. Storage account, datastore, and data asset

These three objects have different responsibilities:

Storage account	→ physically stores files
Datastore	→ connects Azure ML to storage
Data asset	→ named and versioned reference to data

A training job normally uses a data asset without needing to know the full physical storage path or credentials.

2. Data versioning

A data asset version represents one specific state of the data.

```
foliascan-tomato-images:1
foliascan-tomato-images:2
```

When the content or preparation changes, a new version should normally be registered instead of silently replacing the existing version.

3. Data lineage

Data lineage records which data version was used by a job, experiment, or model.

This makes it possible to answer:

Which exact dataset produced this model?

4. Separation of data and metadata

FoliaScan keeps three separate assets:

- Tomato images
- Final leakage-safe dataset manifest
- Original source manifest

This separates the physical images, FoliaScan's split decisions, and the original PlantVillage metadata.

5. Verification before training

Successful registration does not automatically prove that everything is correct.

Before training, the project should verify:

- Asset names and versions
- Asset types
- Required metadata and tags
- Local image and manifest counts
- Integrity fingerprints

Key takeaway

Phase 3 turns local files into identifiable, versioned, reusable ML inputs. This is an important MLOps step because cloud training should depend on explicit dataset versions rather than changing local folders.

- 3.1 Inspect storage and the default datastore

Objective

Understand the existing Azure ML storage environment before uploading or registering the FoliaScan dataset.

What was implemented

The workspace contained four managed datastores:

```
workspaceblobstore
workspaceartifactstore
workspacefilestore
workspaceworkingdirectory
```

workspaceblobstore was selected for the FoliaScan image and manifest assets.

The local dataset was also measured:

```
Images: 18,160
Approximate size: 0.52 GB
```

No new storage account or datastore was required, and no data or compute node was activated during this inspection.

Concepts I should understand

1. Storage account

A storage account is the Azure resource that physically stores files and objects.

The Azure ML workspace uses storage for items such as:

- Training data
- Job outputs
- Model artifacts
- Logs
- Temporary files

2. Datastore

A datastore is an Azure ML connection to a storage location.

It allows Azure ML jobs to access data without hardcoding storage credentials into training code.

A datastore is therefore a **connection**, not the dataset itself.

3. Default datastore

The default datastore is the storage location Azure ML normally uses when a command uploads local files without specifying another destination.

For FoliaScan:

Default Blob datastore → workspaceblobstore

4. Storage and compute are separate

Listing datastores or inspecting data assets is a control-plane operation.

It does not:

- Start CPU or GPU nodes
- Run training
- Load the dataset into memory

Important note

The existing workspace-managed storage was sufficient. Creating an additional storage account without a clear reason would add complexity and possibly extra cost.

Essential commands

```
az ml datastore list
az ml datastore show `
  --name workspaceblobstore
az ml data list
```

Review questions

1. What is the difference between a storage account and a datastore?
2. What is the purpose of the default datastore?
3. Why was workspaceblobstore suitable for FoliaScan?
4. Why should dataset size be checked before uploading?
5. Does listing datastores start an Azure compute cluster?

Key takeaway

Inspect existing storage before creating anything new. The workspace already had a suitable Blob datastore for the image folder and manifests.

- **3.2 Define the Azure data-asset structure**

Objective

Decide which reusable Azure ML data assets the project needs and how they should be separated.

What was implemented

Three data assets were defined:

`foliascan-tomato-images`

→ `uri_folder`

`foliascan-dataset-manifest`

→ `uri_file`

`foliascan-source-manifest`

→ `uri_file`

Each asset was designed with:

- A clear name
- An explicit version
- An asset type
- A source path
- A description
- Project and content tags

Concepts I should understand

1. Azure ML data asset

A data asset is a named and versioned reference that Azure ML jobs can use as an input.

Instead of using:

Long storage path

a training job can reference:

`foliascan-tomato-images:1`

2. uri_folder

`uri_folder` represents a directory containing multiple files.

It is appropriate for:

18,160 tomato images

Azure ML can mount or download the complete folder for a job.

3. uri_file

uri_file represents one individual file.

It is appropriate for:

dataset_manifest.csv

plantvillage_source_manifest.csv

4. Why the assets are separate

The three assets represent different information:

AssetResponsibility Tomato images | Physical image data

Dataset manifest | Final train, validation, and test decisions

Source manifest | Original PlantVillage metadata

Separating them improves auditability and allows one component to change without treating the entire dataset as one unclear object.

5. Asset version

Version 1 represents the current verified state.

A meaningful change should normally create Version 2, for example:

- New images
- Corrected labels
- New split logic
- Updated manifest rows

Important note

Do not overwrite Version 1 simply because the local files changed. A silent replacement would weaken reproducibility and data lineage.

1. What is an Azure ML data asset?
2. What is the difference between uri_folder and uri_file?
3. Why were the images and manifests registered separately?
4. What should happen when the dataset content changes?
5. Why is a friendly asset name better than using a long storage path directly?

Key takeaway

Design the data-asset structure before uploading. Clear asset boundaries and explicit versions make future training inputs easier to understand and reproduce.

- 3.3 Prepare version-controlled data YAML files

Objective

Describe each Azure ML data asset in a declarative YAML file that can be reviewed and stored in Git.

What was implemented

Three YAML files were created:

```
infra/azure/data/  
├── tomato-images.yml  
├── dataset-manifest.yml  
└── source-manifest.yml
```

The repository currently contains all three definitions.

Concepts I should understand

1. Declarative configuration

A declarative file describes the desired resource:

This data asset should have this name, version, type, path, description, and tags.

Azure ML then performs the required operations to create that result.

2. YAML asset fields

A simplified FoliaScan YAML definition is:

```
$schema: https://azuremlschemas.azureedge.net/latest/data.schema.json  
  
name: foliascan-tomato-images  
version: 1  
type: uri_folder  
  
path: ../../../../data/raw/plantvillage_tomato_color  
  
description: PlantVillage Tomato image subset used by FoliaScan.  
  
tags:  
  project: foliascan  
  source: plantvillage  
  content: tomato-images
```

The actual tomato-images.yml uses this structure.

3. Schema

The \$schema field tells editors and Azure ML which YAML structure is expected.

It can help identify:

- Unsupported properties
- Incorrect field names
- Invalid asset types
- Structural errors

4. Local path

The path field identifies the local file or folder to upload.

Examples:

```
Image folder → data/raw/plantvillage_tomato_color
Manifest      → data/processed/dataset_manifest.csv
```

The relative path is resolved from the YAML file's location.

5. Tags

Tags provide searchable metadata.

FoliaScan uses tags such as:

```
project: foliascan
source: plantvillage
content: tomato-images
```

Tags help distinguish assets when a workspace contains many datasets.

6. YAML and Git

The YAML definitions belong in Git because they are small configuration files describing the infrastructure.

The actual raw dataset does not belong in Git because it is large and managed separately.

Important note

Preparing a YAML file does not upload anything by itself. Upload and registration happen only when the Azure ML create command is executed in subphase 3.4.

Review questions

1. What does a declarative YAML file describe?
2. What is the purpose of the `$schema` field?
3. What information do name, version, type, and path provide?
4. Why should YAML files be committed while the raw images are excluded?
5. Why are tags useful?

Key takeaway

Version-controlled YAML makes the data-asset definitions reviewable and repeatable. It separates the description of an asset from the command that creates it.

- 3.4 Upload and register versioned data assets

Objective

Upload the local dataset to Azure Storage and register it as reusable Azure ML data assets.

What was implemented

Before uploading, the local inputs were checked:

Images:	18,160
Image-folder size:	about 554 MB
Dataset-manifest rows:	18,160
Source-manifest rows:	18,160

Three Version 1 assets were uploaded to workspaceblobstore and registered:

```
foliascan-tomato-images:1  
→ uri_folder
```

```
foliascan-dataset-manifest:1  
→ uri_file
```

```
foliascan-source-manifest:1  
→ uri_file
```

The two small manifest files were uploaded first to validate the YAML configuration and Azure permissions before transferring the larger image folder.

Concepts I should understand

1. Upload

Upload transfers the physical files from the local computer to Azure Storage.

Local files → workspaceblobstore

2. Registration

Registration creates a named and versioned Azure ML reference to the cloud location.

Cloud storage path

↓

```
foliascan-tomato-images:1
```

Upload and registration are related, but they are not the same concept.

3. Local-path behaviour

Because the YAML definitions contain local paths, Azure ML performs two operations:

Upload local content to the default datastore

↓

Register a data asset referencing the cloud content

4. AzureML URI

An AzureML URI is a workspace-aware path referencing data through Azure Machine Learning.

It can identify data using the workspace, datastore, path, asset name, and version instead of exposing storage credentials in the training code.

5. Data lineage in future jobs

When a training job uses:

```
foliascan-tomato-images:1
```

Azure ML can record that Version 1 was used.

If Version 2 is used later, the job history can distinguish the two datasets.

Important note

Uploading or registering data does not start the CPU or GPU clusters. However, cloud storage and data transfer are separate Azure resources and may still have costs.

Essential commands

```
az ml data create `
--file infra/azure/data/dataset-manifest.ymlaz ml data create `
--file infra/azure/data/source-manifest.ymlaz ml data create `
--file infra/azure/data/tomato-images.ymlaz ml data list
```

Review questions

1. What is the difference between upload and registration?
2. What happens when `az ml data create` receives a local path?
3. Why were the manifests uploaded before the image folder?

4. What does an AzureML URI represent?
5. How does asset versioning support data lineage?
6. Does uploading a data asset start the training compute?

Key takeaway

Physical files are uploaded to storage, while registration creates the friendly, versioned Azure ML object that future jobs will use.

- 3.5 Verify data integrity and Azure access

Objective

Confirm that the registered assets are accessible and that their metadata matches the verified local dataset.

What was implemented

A read-only Python verification command confirmed:

```
Verification status:    verified
Registered assets:     3
Local images:          18,160
Dataset-manifest rows: 18,160
Source-manifest rows:  18,160
```

For each Azure data asset, the tool checked:

- Correct name
- Correct Version 1
- Correct asset type
- Required tags
- Presence of a cloud storage path

It also calculated SHA-256 fingerprints for the two local manifest files.

The verification did not upload, modify, download, or delete data and did not start a compute cluster.

Concepts I should understand

1. Data integrity

Data integrity means confidence that data is complete, consistent, and has not changed unexpectedly.

Useful checks include:

- File counts
- Manifest row counts
- Metadata
- Content hashes

2. SHA-256 fingerprint

SHA-256 calculates a fingerprint from file content.

```
File content
  ↓
SHA-256
  ↓
Fixed-length fingerprint
```

Changing even a small part of the file normally produces a different fingerprint.

A hash is stronger evidence of unchanged content than checking only the filename or file size.

3. Metadata verification

Metadata verification confirms that Azure registered the expected object:

```
Expected name
Expected version
Expected type
Expected tags
Cloud path exists
```

It checks the asset definition, not the model or training data behaviour.

4. Read-only verification

A read-only command retrieves and validates information without changing cloud resources.

This makes it suitable for checking access and configuration without producing new assets or compute costs.

5. Verification limitation

The verifier checks:

- Azure asset metadata
- Local image and manifest counts
- Local manifest hashes

It does **not** download every Azure image and compare every cloud byte with the local

dataset.

Therefore:

```
Metadata and local integrity verified
#
Full byte-for-byte cloud dataset verification
```

This limitation is important to understand. The repository documentation explicitly describes the command as read-only and lists the exact metadata and local checks it performs.

Important note

Registration should always be followed by verification. A successful create command alone does not prove that the correct asset name, type, version, tags, and local dataset counts were used.

Essential command

```
poetry run python -m foliascan.cloud.azure_data_assets `
  --image-root data/raw/plantvillage_tomato_color `
  --dataset-manifest data/processed/dataset_manifest.csv `
  --source-manifest data/processed/plantvillage_source_manifest.csv
```

Optional local checks:

```
Get-FileHash <file> -Algorithm SHA256Import-Csv <manifest.csv>
```

Review questions

1. What does data integrity mean?
2. Why is SHA-256 more useful than checking only the filename?
3. Which metadata fields were verified for each asset?
4. Why should registration be followed by verification?
5. What makes the verification command read-only?
6. What does the verifier not compare byte-for-byte?
7. Why should Azure identifiers remain outside the source code?

Key takeaway

Cloud data should be verified before it is used for training. Verification confirms that the expected assets exist and that their metadata and local source files match the intended dataset state.

✓ Phase 4 — Train the model in Azure

Goal

The goal of Phase 4 is to run the existing FoliaScan training workflow as reproducible Azure Machine Learning command jobs.

Instead of redesigning the model, this phase connects the components already prepared:

```
Source code
+
Versioned data assets
+
Azure ML environment
+
CPU or GPU compute
↓
Azure ML command job
↓
Metrics and training artifacts
```

Complete workflow

```
Select a reproducible training environment
↓
Make the existing training command cloud-compatible
↓
Define a small CPU smoke-test job
↓
Define the full GPU training job
↓
Submit and monitor both jobs
↓
Download and verify metrics and artifacts
```

Main concepts

1. Environment and compute are different

Environment → software required by the job
Compute → hardware on which the job runs

The environment contains Python, PyTorch, CUDA, system libraries, and other dependencies.

The compute target provides CPU, memory, and possibly GPU resources.

2. Azure ML command job

A command job describes:

- Which code to run
- Which command to execute
- Which environment to use
- Which compute target to use
- Which data assets are inputs
- Where outputs should be stored

The job definition is stored in YAML so it can be reviewed and reproduced.

3. Mounted inputs and managed outputs

Azure ML mounts registered data assets into the job container.

```
Registered data asset
    ↓
Read-only mounted path
    ↓
Training program
```

The training program writes generated files to a writable Azure-managed output directory.

4. Cloud job lifecycle

A cloud training job normally passes through several stages:

```
Submit
    ↓
Upload code
    ↓
Queue
    ↓
Provision compute
    ↓
Prepare environment
    ↓
Mount data
    ↓
Run training
    ↓
Save outputs
    ↓
Complete
    ↓
Scale compute down
```

5. Metrics and artifacts

- **Metrics** describe training behaviour, such as loss and accuracy.
- **Artifacts** are generated files, such as checkpoints and training histories.

Both are needed to understand and reproduce a training run.

Key takeaway

Phase 4 proves that the local FoliaScan workflow can run on managed Azure infrastructure using versioned inputs, reproducible software, controlled compute, and Azure-managed outputs.

- 4.1 Select the Azure ML training environment

Objective

Choose a reproducible software environment that can run the FoliaScan PyTorch workflow on both CPU and GPU compute.

What was implemented

FoliaScan uses the Microsoft-curated environment:

`AzureML-acpt-pytorch-2.8-cuda12.6:15`

The same exact environment version is referenced by both the CPU and GPU job definitions.

Concepts I should understand

1. Azure ML environment

An Azure ML environment defines the software required by a job, including:

- Operating system
- Python runtime
- Python packages
- PyTorch
- CUDA libraries
- System dependencies

It helps the same code run consistently on Azure compute.

2. Curated environment

A curated environment is prepared and maintained by Microsoft.

Benefits include:

- Faster setup than building a new image
- Common ML libraries already installed
- Tested PyTorch and CUDA combinations
- Reduced container-maintenance work

3. Custom environment

A custom environment is needed when the curated environment does not contain the required packages or system configuration.

It can be defined using:

- A Conda specification
- A Docker image
- A Dockerfile and additional dependencies

For this phase, the curated environment was sufficient, so creating a custom one would add unnecessary complexity.

4. Container image

The environment runs as a container image on Azure compute.

The container packages the runtime so the training job does not depend on software installed manually on the compute node.

5. Environment version pinning

Using:

```
AzureML-acpt-pytorch-2.8-cuda12.6@latest
```

could resolve to a different environment in the future.

Using:

```
AzureML-acpt-pytorch-2.8-cuda12.6:15
```

pins one exact version and improves reproducibility.

Important note

The Azure environment is independent of the local Poetry environment. They may contain compatible packages, but Azure does not automatically install the local poetry.lock inside the curated container.

Review questions

1. What is the difference between an environment and a compute target?
2. What does an Azure ML environment contain?
3. What is the difference between curated and custom environments?
4. Why was a curated environment suitable for the first Azure run?
5. Why is an exact environment version safer than @latest?

Key takeaway

The environment controls the software runtime, while compute provides the hardware.
Pinning the environment version makes future runs more reproducible.

- 4.2 Adapt the training entry point for cloud inputs

Objective

Make the existing local training command work inside Azure ML without creating a separate cloud-only training script.

What was implemented

The same entry point is used locally and in Azure:

```
python -m foliascan.training.train
```

It accepts:

```
--data-dir  
--manifest  
--config  
--output-dir
```

The input paths can be:

- Local relative paths
- Local absolute paths
- Absolute Linux mount paths created by Azure ML

The output artifacts are written below the supplied `--output-dir`:

```
best_model.pt  
last_model.pt  
history.csv  
history.json
```

The implementation also supports limited-batch options for cloud smoke tests:

```
--max-train-batches  
--max-validation-batches
```

The training pipeline continues to use only the train and validation splits; the test split remains excluded.

Concepts I should understand

1. Portable training entry point

A portable entry point can run in different environments without changing its core training

logic.

```
Local execution
    ↓
Same training module
    ↑
Azure execution
```

This avoids maintaining two separate implementations that may gradually behave differently.

2. Path portability

Azure jobs run inside Linux containers and provide paths such as:

```
/mnt/azureml/...
```

The program must use the supplied paths directly rather than assume that all data exists relative to the local repository root.

3. Separation of code and configuration

The Python module contains the training logic.

The configuration file contains settings such as:

- Batch size
- Learning rate
- Model choice
- Early-stopping patience

The Azure job YAML decides how the training program is executed in the cloud.

4. Managed output directory

The training program should not choose an arbitrary local output path inside the container.

It writes to the Azure-provided output directory so Azure can persist the artifacts after the compute node stops.

5. One source of training logic

Reusing the existing entry point keeps the following in one implementation:

- Dataset loading
- Transforms
- Model construction
- Training loop
- Validation
- Checkpointing
- Artifact writing

The repository documentation explicitly states that a separate Azure training program was not created.

Important note

Cloud compatibility should not change dataset splitting or model-selection rules. Azure changes where the workflow runs, not how train, validation, and test data are used.

Review questions

1. Why is one portable training entry point preferable to separate local and Azure scripts?
2. Why must the program support absolute Linux paths?
3. What is the role of `--output-dir`?
4. Why are limited-batch arguments useful?
5. Why must the test split remain excluded from the training job?

Key takeaway

The local training workflow was made portable rather than rewritten. Azure supplies paths and infrastructure, while the core ML behaviour remains the same.

- 4.3 Define a CPU smoke-test job

Objective

Create a small and low-cost Azure job that validates the entire cloud training pipeline before using GPU compute.

What was implemented

The job is defined in:

```
infra/azure/jobs/cpu-smoke-test.yml
```

Main configuration:

Setting | Value

Display name | foliascan-cpu-smoke-test

Experiment | foliascan-training

Compute | cpu-fs-dev

Device | CPU

Epochs | 1

Training batches | 2

Validation batches | 1

Timeout | 1,800 seconds

Registered inputs:

```
foliascan-tomato-images:1
```

```
foliascan-dataset-manifest:1
```

Named output:

```
training_output
```

The YAML uses read-only data mounts and a writable managed output.

Concepts I should understand

1. Command-job YAML

The CPU YAML defines the complete execution contract:

Code

Command

Environment

Compute

Inputs

Outputs
Limits
Tags

2. Code snapshot

The job uses:

```
code: ../../..
```

Azure uploads the relevant repository content as a job code snapshot.

This allows the remote compute node to run the project code without directly reading the GitHub repository.

3. Read-only mount

The data inputs use:

```
mode: ro_mount
```

The job can read the registered images and manifest but cannot modify the original assets.

4. Writable output mount

The output uses:

```
mode: rw_mount
```

Azure provides a writable directory and persists its contents in workspace storage after the job completes.

5. Limited-batch smoke test

Only two training batches and one validation batch are processed.

The purpose is to detect integration failures such as:

- Incorrect paths
- Missing packages
- Invalid job configuration
- Data-mounting failures
- Compute problems
- Artifact-writing failures

It is not intended to measure model quality.

6. Timeout

The job timeout prevents a small validation job from running indefinitely because of a problem.

Important note

The CPU smoke-test loss and accuracy are not meaningful model-performance results because only a few batches were processed.

Review questions

1. What does a command-job YAML define?
2. What is a code snapshot?
3. What is the difference between ro_mount and rw_mount?
4. Why does the CPU job process only a few batches?
5. Which failures can a cloud smoke test detect?
6. Why should its accuracy not be reported as model performance?

Key takeaway

The CPU job validates the complete cloud path at low cost before the project commits to a longer GPU training run.

- 4.4 Define the GPU training job

Objective

Define the complete ResNet18 training job using Azure GPU compute and the full train and validation datasets.

What was implemented

The job is defined in:

`infra/azure/jobs/gpu-training.yml`

Main configuration:

Setting | Value

Display name | foliascan-gpu-training

Experiment | foliascan-training

Compute | gpu-fs-dev

Device | CUDA

Epochs | 10

Batch size | 32

Optimizer | AdamW

Learning rate | 0.001

Weight decay | 0.0001

Timeout | 7,200 seconds

It uses the same:

- Source code
- Training entry point
- Environment version
- Registered image asset
- Registered manifest asset
- Named output

Unlike the smoke test, it does not limit the number of train or validation batches.

Concepts I should understand

1. Full training job

A full training job processes all configured train and validation batches for each epoch.

Its metrics can support checkpoint selection, unlike smoke-test metrics.

2. CUDA device

The job explicitly passes:

```
--device cuda
```

This ensures the training program fails clearly if the GPU runtime is unavailable instead of silently falling back to CPU.

3. CPU smoke test versus GPU training

CPU smoke test	GPU full training	Validates integration Trains the real baseline
CPU compute	GPU compute	
One epoch	Ten epochs	
Limited batches	All batches	
Low cost	Higher cost	
Metrics not meaningful	Validation metrics useful	

4. Same inputs, different workload

Both jobs use the same Version 1 data assets.

This means differences between the two jobs come mainly from execution scale and compute rather than different dataset versions.

5. Job tags

Tags such as the following make cloud jobs easier to identify:

```
project=foliascan  
stage=gpu-full-training  
data-version=1  
epochs=10
```

Important note

The GPU job still performs training and validation only. It does not run final test-set evaluation, register the model, or deploy it.

Review questions

1. What is the main difference between the CPU and GPU jobs?
2. Why does the GPU command explicitly request CUDA?

3. Why should both jobs use the same data-asset versions?
4. Why are exact environment and dataset versions important?
5. What responsibilities remain outside this training job?

Key takeaway

The GPU YAML scales the already validated workflow to full training without changing its core data, model, or artifact logic.

- 4.5 Submit and monitor the training job

Objective

Submit the CPU and GPU command jobs, monitor their execution, inspect their logs, and confirm successful completion.

What was implemented

The execution order was:

```
Submit CPU smoke test
    ↓
Confirm CPU status = Completed
    ↓
Submit GPU training
    ↓
Stream GPU logs
    ↓
Confirm GPU status = Completed
```

Azure uploaded a code snapshot of approximately 1.65 MB during job submission.

Both jobs completed successfully:

Job	Job name	Status	Compute
-----	----------	--------	---------

CPU smoke test	heroic_pump_6kvy3nnvpt	Completed	cpu-fs-dev
----------------	------------------------	-----------	------------

GPU training	cyan_boot_v9vg1214b3	Completed	gpu-fs-dev
--------------	----------------------	-----------	------------

The GPU job completed all ten epochs.

Concepts I should understand

1. Job submission

The YAML definition becomes an actual Azure ML job when submitted with:

```
az ml job create `
  --file infra/azure/jobs/cpu-smoke-test.yml
```

The returned job name is the unique identifier used for later operations.

2. Job status

A job can pass through states such as:

Queued
Preparing
Running
Completed
Failed
Canceled

The final status must be checked directly rather than inferred from the local terminal.

3. Log streaming

Live logs are viewed using:

```
az ml job stream --name <job-name>
```

Log streaming is only a connection to the running job.

If the local stream is interrupted, the Azure job may continue running.

4. Status checking

The reliable way to inspect the job is:

```
az ml job show `
  --name <job-name> `
  --query status `
  --output tsv
```

5. CPU gate before GPU

The GPU job was submitted only after the CPU smoke test reached Completed.

This prevents spending GPU resources on a workflow that has not passed basic integration checks.

6. Cold-start overhead

The first GPU epoch took much longer than later epochs.

The first epoch included additional work such as:

- Compute startup
- Environment preparation
- Initial data access
- Container startup
- Framework warm-up

Later epochs mainly reflected actual training time.

Essential commands

```
# Submit
az ml job create --file <job-yaml>

# Stream logs
az ml job stream --name <job-name>

# Check final status
az ml job show `
  --name <job-name> `
  --query status `
  --output tsv
```

Important note

Stopping or losing az ml job stream does not automatically cancel the remote job. Job status and cancellation are separate operations.

Review questions

1. What happens when an Azure ML job is submitted?
2. Why is the returned job name important?
3. What is the difference between log streaming and job execution?
4. Why must status be checked separately?
5. Why was the GPU job submitted only after the CPU job completed?
6. Why can the first epoch take much longer than later epochs?

Key takeaway

Cloud training is asynchronous. Submission, log streaming, status checking, and artifact retrieval are separate operations that use the job identifier.

- 4.6 Track metrics and artifacts

Objective

Retrieve the Azure job outputs, inspect training metrics, identify the best checkpoint, and verify that the required artifacts were generated.

What was implemented

The CPU and GPU job names were saved locally:

```
artifacts/azure/job-records/  
├── cpu-job-name.txt  
└── gpu-job-name.txt
```

The named outputs were downloaded into:

```
artifacts/  
├── azure/  
│   └── jobs/  
│       ├── <job-name>/  
│       │   ├── named-outputs/  
│       │   └── training_output/
```

Both jobs produced:

```
best_model.pt  
last_model.pt  
history.csv  
history.json
```

All four required GPU artifacts were verified. The two checkpoint files were approximately 128.1 MB each.

Concepts I should understand

1. Named output

The YAML defines:

```
training_output
```

A named output allows Azure and the user to identify and retrieve one logical group of job-generated files.

2. Artifact download

Artifacts were downloaded with:

```
az ml job download `
  --name <job-name> `
  --output-name training_output `
  --download-path artifacts/azure/jobs/<job-name>
```

Downloading an artifact creates a local copy; the Azure-stored version remains available.

3. Training history

history.csv contains one row of metrics for each epoch and is convenient for:

- Pandas
- Excel
- Plotting
- Manual review

history.json stores the history in a structured format suitable for:

- Scripts
- APIs
- Automation
- Further processing

4. Best and last checkpoints

- best_model.pt contains the epoch with the lowest validation loss.
- last_model.pt contains the final completed epoch.

For this Azure GPU run, both corresponded to epoch 10 because epoch 10 produced the best validation loss.

5. Azure GPU training result

The main results were:

Metric | Result

Completed epochs | 10

Best epoch | 10

Best validation loss | 0.212930

Best validation accuracy | 92.8964%

Final training accuracy | 94.1544%

Early stopped | No

Validation accuracy temporarily dropped during epoch 8 but recovered in epochs 9 and 10.

6. Validation metrics are not test metrics

The reported 92.8964% is validation accuracy.

It is used to understand training and select the checkpoint. It is not final test accuracy.

7. Artifact integrity

File presence, size, and SHA-256 hashes can be checked to confirm that artifacts were downloaded and did not change unexpectedly.

8. Metrics files versus MLflow

In Phase 4:

```
Training code
  ↓
history.csv / history.json
  ↓
Manual inspection
```

In Phase 5:

```
Training run
  ↓
MLflow logging
  ↓
Central experiment comparison
```

Phase 4 saves and reviews metrics, but does not yet provide full MLflow experiment tracking.

Important note

The validation accuracy from this run should not be presented as final model performance. Final performance must come from evaluation of the selected checkpoint on the untouched test split.

Review questions

1. What is a named Azure ML output?
2. What is the difference between metrics and artifacts?
3. What information is stored in history.csv and history.json?
4. What is the difference between best_model.pt and last_model.pt?
5. Why did both checkpoints correspond to epoch 10 in this run?

6. Why is validation accuracy not the same as test accuracy?
7. What remains to be added in Phase 5 with MLflow?

Key takeaway

A completed job is useful only when its metrics, checkpoints, and metadata can be retrieved and interpreted. Phase 4 produced a verified Azure-trained checkpoint, while central experiment tracking remains the next step.

✓ Phase 5 — Track experiments with MLflow

Goal

The goal of Phase 5 is to make training experiments easier to **track, reproduce, and compare** by logging important run information with MLflow inside Azure Machine Learning.

Phase 4 produced metrics and artifacts, but they had to be inspected manually from files such as history.csv.

Phase 5 adds structured experiment tracking:

```
Training run
  ↓
Parameters + metrics + artifacts
  ↓
MLflow
  ↓
Azure ML experiment
  ↓
Compare runs
```

Complete workflow

```
Inspect Azure ML's MLflow integration
  ↓
Add optional MLflow support to the training code
  ↓
Log parameters, metrics, and useful artifacts
  ↓
Enable tracking in Azure job YAML
  ↓
Run a real tracked Azure job
  ↓
Compare multiple runs
```

Main concepts

1. Experiment and run

An **experiment** groups related executions.

```
Experiment: foliascan-training
├── Run 1
├── Run 2
└── Run 3
```

A **run** is one execution of the training workflow with a particular configuration.

For example, two runs may differ in:

- Learning rate
- Number of epochs
- Batch size
- Model configuration

2. Parameters, metrics, and artifacts

MLflow tracks three main types of information:

Type | Meaning

Parameters | Configuration of the run

Metrics | Numeric values produced during training

Artifacts | Files generated or saved for analysis

Example:

```
Parameter → learning_rate = 0.001
Metric    → validation_loss = 0.21
Artifact  → history.csv
```

3. Tracking backend

The **tracking backend** is where MLflow stores experiment information.

For Azure ML jobs, the Azure ML workspace acts as the MLflow tracking backend.

The submitted job already has a run identity, so the training code can log information to that existing Azure run.

4. Metric history

Metrics should not only store a final number.

For training, they are logged by epoch:

```
Epoch 1 → validation_loss
Epoch 2 → validation_loss
Epoch 3 → validation_loss
```

This creates curves that show how the model changes during training.

5. Experiment tracking is not model registration

MLflow tracking

→ What happened during training?

Model registration

→ Which trained model should become a managed model asset?

Tracking helps analyse experiments. Registration is a later lifecycle step.

Key takeaway

Phase 5 turns isolated training results into structured experiment records. Instead of manually comparing files from different runs, MLflow connects configuration, metric history, and artifacts to each Azure ML run.

- 5.1 Inspect the Azure MLflow tracking setup

Objective

Understand how Azure Machine Learning and MLflow work together before changing the training code.

What was implemented

The existing Azure setup was inspected and confirmed that:

Azure ML workspace

↓

MLflow tracking backend

Experiment:

foliascan-training

↓

Existing CPU and GPU runs

The previous Phase 4 Azure jobs were already grouped under the foliascan-training experiment.

MLflow was not yet installed in the local FoliaScan environment.

Concepts I should understand

1. MLflow experiment

An experiment is a logical collection of related runs.

For example:

```
foliascan-training
├── CPU smoke test
├── GPU baseline
└── Future experiments
```

This makes it easier to compare different executions of the same project.

2. MLflow run

A run represents one execution.

A run can contain:

Parameters
Metrics
Artifacts
Metadata

3. Azure ML job and MLflow run

When an Azure ML command job starts, Azure already creates a run context.

Therefore FoliaScan does not need to create another separate MLflow run manually.

The training code did not need:

```
mlflow.set_tracking_uri(...)  
mlflow.set_experiment(...)  
mlflow.start_run(...)
```

4. Tracking context

The tracking context tells MLflow:

Which workspace, experiment, and run should receive these logs?

Azure ML provides that context automatically for submitted jobs.

Important note

Creating an additional MLflow run inside an existing Azure ML job could produce duplicate or disconnected run records.

Review questions

1. What is the difference between an MLflow experiment and a run?
2. What acts as the MLflow tracking backend for FoliaScan?
3. Why does an Azure ML job already have a run identity?
4. Why was `mlflow.start_run()` not needed inside the Azure job?
5. What problem could duplicate MLflow runs cause?

Key takeaway

Azure ML already manages the run lifecycle. MLflow should attach tracking information to

that existing run rather than create a second unrelated run.

- 5.2 Add MLflow dependencies and tracking support

Objective

Add MLflow support to the existing training pipeline while keeping it optional for normal local training.

What was implemented

The project added:

```
mlflow = 2.16.2
azureml-mlflow = 1.62.0.post5
```

This updated:

```
pyproject.toml
poetry.lock
```

A helper module was added:

```
src/foaliascan/training/mlflow_tracking.py
```

The training CLI also received:

```
--enable-mlflow
```

Without this flag, MLflow tracking remains disabled.

Concepts I should understand

1. mlflow

The main MLflow package provides APIs for:

- Logging parameters
- Logging metrics
- Logging artifacts
- Working with experiments and runs

2. azureml-mlflow

This package connects MLflow with Azure Machine Learning so MLflow data can appear in the Azure ML workspace.

3. Opt-in tracking

Tracking is explicitly enabled with:

```
--enable-mlflow
```

This keeps the normal local workflow unchanged.

Local training

→ MLflow disabled by default

Azure tracked training

→ --enable-mlflow

4. Manual logging versus autologging

MLflow can automatically capture information using autologging.

FoliaScan instead uses **manual logging**.

Manual logging gives explicit control over:

- Which parameters are stored
- Metric names
- Metric steps
- Final summary metrics
- Which artifacts are uploaded

This is useful in a learning project because the tracking behaviour remains clear.

5. Tracking should not change training behaviour

Adding MLflow should observe the training process rather than change:

- Model architecture
- Data splitting
- Optimizer behaviour
- Checkpoint selection
- Training results

Important note

Tracking is optional so a simple local training run does not unexpectedly create cloud tracking records.

Review questions

1. What is the role of mlflow?
2. What does azureml-mlflow add?
3. Why is MLflow tracking opt-in?
4. What is the difference between manual logging and autologging?
5. Why should tracking remain separate from the core training logic?

Key takeaway

Experiment tracking should be explicit and modular. FoliaScan added MLflow around the existing training workflow instead of embedding cloud-specific behaviour throughout the training code.

- 5.3 Log parameters, epoch metrics, and artifacts

Objective

Record enough information from each training run to understand how it was configured, how it behaved, and what files it produced.

What was implemented

Parameters logged

Examples include:

```
model_name
epochs
batch_size
learning_rate
optimizer_name
weight_decay
image_size
pretrained
freeze_backbone
random_seed
requested_device
```

These describe how the run was configured.

Per-epoch metrics

After every epoch:

```
train_loss
train_accuracy
validation_loss
validation_accuracy
learning_rate
elapsed_seconds
```

The epoch is used as the metric step:

```
step = epoch
```

Final summary metrics

Examples include:

```
completed_epochs
best_epoch
```

```
best_validation_loss
best_validation_accuracy
early_stopped
```

Lightweight artifacts

MLflow stores items such as:

```
config/
history/
```

including:

```
training.example.yaml
history.csv
history.json
```

The large checkpoint files remain in the Azure ML training_output rather than being uploaded again to MLflow.

Concepts I should understand

1. Parameters

Parameters describe the configuration chosen **before or during setup of the run**.

Examples:

```
batch_size = 32
learning_rate = 0.001
epochs = 10
```

They make it possible to understand why two runs differ.

2. Metrics

Metrics describe numerical behaviour or performance.

Examples:

```
train_loss
validation_loss
validation_accuracy
```

Metrics can change over time.

3. Metric step

The step identifies where a metric belongs in a sequence.

For FoliaScan:

```
step 1 → epoch 1  
step 2 → epoch 2  
...
```

Without steps, MLflow would know the metric values but would have less information about how they evolved.

4. Artifacts

Artifacts are files associated with the run.

Examples:

```
Configuration file  
Training history  
Plots  
Reports
```

5. Avoiding artifact duplication

Phase 4 already stores:

```
best_model.pt  
last_model.pt
```

inside the Azure named output.

Uploading the same approximately 128 MB checkpoints again to MLflow would duplicate storage without adding value.

Therefore:

```
MLflow  
→ lightweight experiment artifacts
```

```
Azure named output  
→ large model checkpoints
```

Important note

MLflow tracking does not replace Azure job outputs. The two mechanisms can store different types of artifacts for different purposes.

Review questions

1. What is the difference between a parameter and a metric?
2. Why are metrics logged with an epoch step?
3. What are MLflow artifacts?
4. Why are history.csv and the configuration useful artifacts?
5. Why were model checkpoints not uploaded again to MLflow?
6. How is MLflow tracking different from model registration?

Key takeaway

A useful tracked run should explain three things:

How was it configured? → Parameters
How did it behave? → Metrics
What did it produce? → Artifacts

- 5.4 Update the Azure ML job definitions

Objective

Enable MLflow tracking in Azure jobs through version-controlled job YAML rather than manual portal configuration.

What was implemented

Both Azure job definitions were updated:

```
infra/azure/jobs/cpu-smoke-test.yml
infra/azure/jobs/gpu-training.yml
```

Their commands now include:

```
--enable-mlflow
```

The existing infrastructure remained unchanged:

- Registered data assets
- Compute target
- Pinned environment
- Named output
- Timeouts
- Training configuration

The experiment remains:

```
foliascan-training
```

Concepts I should understand

1. Job YAML as the execution contract

The Azure job YAML describes how the cloud workload should execute:

```
Code
Command
Environment
Compute
Data inputs
Outputs
Experiment
Tracking option
```

2. Tracking as configuration

MLflow tracking is enabled through:

```
--enable-mlflow
```

inside version-controlled YAML.

This is preferable to manually enabling settings in the portal because it is:

- Repeatable
- Reviewable
- Stored in Git
- Easier to reproduce

3. Infrastructure versus application behaviour

Most of the Azure job did not change.

Only tracking behaviour was enabled.

```
Same training job
+
MLflow flag
=
Tracked training job
```

Important note

When adding experiment tracking, avoid changing unrelated compute, data, or model settings at the same time. This makes it easier to verify that any differences come from the tracking integration rather than from a changed workload.

Review questions

1. Why was MLflow enabled through the YAML file?
2. What role does the job YAML play in Azure ML?
3. Which job properties remained unchanged?
4. Why is version-controlled configuration preferable to portal-only changes?
5. Why should unrelated training settings remain unchanged when adding tracking?

Key takeaway

The Azure job definition should make tracking behaviour explicit. Anyone reviewing the YAML can see that the job runs the same training workflow with MLflow enabled.

- 5.5 Run and inspect tracked experiments

Objective

Run a real Azure ML job and confirm that parameters, metrics, and artifacts are correctly recorded by MLflow.

What was implemented

A tracked CPU smoke-test job completed successfully:

```
Display name: foliascan-cpu-smoke-test
Experiment:   foliascan-training
Compute:      cpu-fs-dev
Status:       Completed
```

Configuration:

```
Epochs:           1
Batch size:         32
Maximum training batches: 2
Maximum validation batches: 1
Device:             CPU
Learning rate:      0.001
Optimizer:          AdamW
```

Azure ML Studio showed:

- Parameters
- Metrics
- Metric steps
- Configuration artifacts
- History artifacts
- Git metadata
- Registered input data assets
- Environment information

No model was registered in this phase.

Concepts I should understand

1. Validation of the tracking pipeline

This run was designed to answer:

Does MLflow tracking work correctly inside a real Azure ML job?

It was not designed to answer:

Is this an accurate model?

2. Training time versus total job time

MLflow recorded training time of approximately:

9.15 seconds

But the total Azure job took longer because Azure also needed to:

- Start compute
- Prepare the environment
- Mount data
- Set up code
- Start the container

Therefore:

Training time $\neq$ Total cloud job duration

3. Tracking verification

Successful experiment tracking means checking that:

Parameters exist
Metrics exist
Steps are correct
Artifacts exist
Run metadata is connected

A completed Azure job alone does not prove that MLflow logged everything correctly.

Important note

The smoke-test validation metrics were poor because only a few batches were processed.

They are **not model-quality results**.

Their purpose is to verify the MLflow integration.

Review questions

1. What was the main purpose of this tracked CPU run?

2. Why are its validation metrics not meaningful for model selection?
3. What should be inspected to confirm MLflow is working?
4. Why is total Azure job duration longer than training time?
5. What metadata did Azure ML preserve alongside the MLflow results?

Key takeaway

A tracking implementation should be tested with a real cloud job. The goal of this smoke test was to prove that MLflow records reached Azure correctly, not to train the best model.

- 5.6 Compare runs and document the results

Objective

Use MLflow to compare multiple runs with different configurations and understand the difference between tracking validation and real model comparison.

What was implemented

Two tracked CPU runs were compared.

Run 1

Epochs: 1
Maximum training batches: 2
Maximum validation batches: 1

Run 2

Epochs: 2
Maximum training batches: 5
Maximum validation batches: 2

The second run produced two metric steps:

Step 1
Step 2

Azure ML allowed the runs to be compared using:

- Parameters
- Metric histories
- Final metrics
- Runtime
- Artifacts

Both runs remained intentionally small CPU tests.

Concepts I should understand

1. Run comparison

MLflow allows multiple runs under the same experiment to be compared.

For example:

Run | Epochs | LR | Batch size | Validation loss

Run A | 10 | 0.001 | 32 | ...

Run B | 10 | 0.0005 | 32 | ...

This is much easier than manually opening several history.csv files.

2. Fair experiment comparison

For a meaningful model comparison, runs should use:

- The same train/validation data
- The same evaluation protocol
- Comparable compute conditions when relevant
- Controlled changes

Ideally, only the variables being investigated should change.

For example:

Run A: `learning_rate = 0.001`

Run B: `learning_rate = 0.0005`

Everything else stays the same.

3. Pipeline validation versus model evaluation

These are different goals.

Pipeline validation

Few batches

Short runtime

Low cost

Goal → Does the system work?

Model-quality experiment

Full dataset

Complete training

Controlled hyperparameters

Goal → Which configuration performs better?

4. Metric curves

Step-based logging makes it possible to compare learning behaviour rather than only final numbers.

Two runs may finish with similar accuracy but have very different:

- Convergence speed
- Validation stability
- Loss curves
- Overfitting patterns

Important note

The second CPU run performing worse than the first is not meaningful evidence about the model.

Both used only a few batches. They were created to test run comparison, not model selection.

Review questions

1. Why should related runs belong to the same experiment?
2. What information can MLflow compare between runs?
3. What makes an experiment comparison fair?
4. Why should only controlled variables change between experiments?
5. What is the difference between pipeline validation and model-quality evaluation?
6. Why are metric curves more useful than only final metrics?

Key takeaway

MLflow makes experiment comparison systematic, but the tracking tool does not make an experiment scientifically valid by itself. Good comparison still requires consistent data, evaluation, and controlled changes.

✓ Phase 6 — Register the best model

Goal

The goal of Phase 6 is to select the correct trained checkpoint, validate that it is safe to use for inference, and register it as a **versioned Azure ML model asset**.

Training creates model files. Registration turns one selected model artifact into a managed asset that can later be deployed.

Complete workflow

```
Select the best checkpoint
      ↓
Validate it for inference
      ↓
Define the model asset in YAML
      ↓
Register the model in Azure ML
      ↓
Verify version, metadata, and lineage
      ↓
Prepare the deployment handoff
```

Main concepts

1. Checkpoint selection

The most recent checkpoint is not automatically the best model.

A clear selection criterion should be used.

For FoliaScan:

```
Selection criterion → lowest validation loss
Selected artifact   → best_model.pt
```

2. Model validation before registration

Before registration, the checkpoint should be checked to ensure that:

- The expected architecture can be reconstructed.
- All model weights are present.
- The class mapping is available.
- The model can run in inference mode.

- The artifact can be uniquely identified.

3. Model asset and version

A registered model becomes a managed Azure ML asset:

`foliascan-tomato-resnet18:1`

Future approved models can become:

`foliascan-tomato-resnet18:2`

`foliascan-tomato-resnet18:3`

without replacing Version 1.

4. Model lineage

A registered model should remain traceable to the training process that created it:

```
Training job
  ↓
Named output
  ↓
best_model.pt
  ↓
Registered model
```

5. Tracking and registration are different

MLflow tracking

→ describes experiments and runs

Model registration

→ creates a managed, versioned model asset

A tracked run is not automatically a registered deployment candidate.

Key takeaway

Phase 6 converts a selected training artifact into a validated, traceable, versioned Azure ML model asset that is ready for the deployment phase.

- 6.1 Select the model candidate

Objective

Select the correct checkpoint for registration using an explicit model-selection criterion.

What was implemented

The selected candidate came from the full Azure GPU training job:

Source job:	cyan_boot_v9vg1214b3
Display name:	foliascan-gpu-training
Checkpoint:	best_model.pt
Architecture:	ResNet18
Classes:	10
Best epoch:	10
Best validation loss:	0.212930
Best validation accuracy:	0.928964

Planned model asset:

Name:	foliascan-tomato-resnet18
Version:	1
Type:	custom_model

The CPU smoke-test models from Phase 5 were not considered because those runs processed only a few batches and were intended for pipeline validation.

Concepts I should understand

1. Model candidate

A model candidate is a trained artifact being considered for registration or deployment.

Not every checkpoint produced during training should become a registered model.

2. Selection criterion

FoliaScan uses:

Lowest validation loss

The checkpoint selected by this rule is:

best_model.pt

rather than simply using `last_model.pt`.

3. Best checkpoint versus latest checkpoint

`best_model.pt`
→ best validation result

`last_model.pt`
→ most recently completed epoch

They can be the same file state, but they do not have the same meaning.

4. Smoke-test model versus deployment candidate

A smoke-test model only proves that the technical pipeline works.

A deployment candidate should come from meaningful full training.

Important note

The reported 92.90% is **validation accuracy**, not final test accuracy.

Model registration does not justify reusing the test set or changing the model-selection rule.

Review questions

1. Why should the latest checkpoint not automatically be registered?
2. What selection criterion was used for FoliaScan?
3. What is the difference between `best_model.pt` and `last_model.pt`?
4. Why were Phase 5 CPU models excluded?
5. Why is validation accuracy not the same as final test accuracy?

Key takeaway

Model registration should begin with a clear selection rule. For FoliaScan, the full GPU checkpoint with the lowest validation loss was selected.

- 6.2 Validate the checkpoint for inference

Objective

Confirm that the selected checkpoint contains everything required to reconstruct and use the model later during inference.

What was implemented

The local best_model.pt checkpoint was inspected.

Important fields included:

```
epoch
model_name
model_state_dict
optimizer_state_dict
class_to_index
training_config
train_metrics
validation_metrics
best_validation_loss
random_seed
```

The model was reconstructed successfully:

```
Architecture:      resnet18
Classes:           10
Missing state keys: 0
Unexpected state keys: 0
Training mode:     False
```

A SHA-256 fingerprint was also calculated for the selected checkpoint.

Concepts I should understand

1. state_dict

A PyTorch state_dict contains the learned model parameters.

The model architecture is recreated first, then the saved weights are loaded into it.

2. Strict weight loading

FoliaScan used:

```
model.load_state_dict(  
    checkpoint["model_state_dict"],  
    strict=True,  
)
```

strict=True requires the checkpoint keys to match the model exactly.

Therefore:

```
Missing keys = 0  
Unexpected keys = 0
```

provides evidence that the checkpoint matches the reconstructed architecture.

3. Evaluation mode

The model was switched to:

```
model.eval()
```

This changes layers such as batch normalization and dropout to inference behaviour.

4. CPU portability

The checkpoint was loaded using:

```
map_location="cpu"
```

This confirms that a model trained on GPU can still be loaded for CPU inference.

5. Artifact fingerprint

SHA-256 identifies the exact checkpoint file.

If the file content changes, the hash should also change.

This supports traceability between:

```
Validated checkpoint  
    ↓  
Registered model
```

Important note

A .pt file existing on disk is not enough evidence that it is usable. Its structure, architecture compatibility, class mapping, and inference behaviour should be verified.

Review questions

1. What does `model_state_dict` contain?
2. Why is `strict=True` useful?
3. What do zero missing and unexpected keys mean?
4. Why must `model.eval()` be used for inference?
5. Why was the checkpoint loaded on CPU?
6. What does the SHA-256 fingerprint provide?

Key takeaway

Validate the actual checkpoint before registration. A deployment asset should be reconstructable, compatible, and traceable.

- 6.3 Prepare a version-controlled model YAML

Objective

Define the Azure ML model asset as version-controlled configuration rather than registering it manually through the Azure portal.

What was implemented

The following file was created:

```
infra/azure/models/foliascan-tomato-resnet18.yml
```

Main fields:

```
name: foliascan-tomato-resnet18
version: 1
type: custom_model
```

The model path references the existing GPU job output:

```
azureml://jobs/cyan_boot_v9vg1214b3/
outputs/training_output/paths/best_model.pt
```

The YAML also records metadata such as:

- Architecture
- Source job
- Selection criterion
- Best epoch
- Validation metrics
- Dataset versions
- Environment version
- Random seed
- Checkpoint hash

Concepts I should understand

1. Model YAML

The YAML describes which model artifact Azure should register and how the asset should be identified.

It makes registration:

- Reviewable

- Repeatable
- Version controlled

2. custom_model

FoliaScan registers the checkpoint as:

```
custom_model
```

because best_model.pt is a raw PyTorch checkpoint.

It is not an MLflow model package containing files such as:

```
MLmodel  
conda.yaml  
python_env.yaml
```

3. Model metadata

Useful metadata connects the registered asset to its training context.

Examples:

```
framework: pytorch  
architecture: resnet18  
source-job: cyan_boot_v9vg1214b3  
best-epoch: 10  
dataset-asset: foliascan-tomato-images:1
```

4. Explicit model versioning

Version 1 is preserved permanently as one model state.

A future approved checkpoint should normally become a new version rather than replacing the existing asset.

5. .gitignore scope

The project originally used:

```
models/
```

which also matched the nested:

```
infra/azure/models/
```

It was changed to:

`/models/`

so only the root-level local model directory is ignored while Azure model YAML files remain tracked in Git.

Important note

The YAML belongs in Git. The large binary model artifact does not need to be committed because Azure already stores it as a training output.

Review questions

1. Why define the model in YAML instead of only using the portal?
2. Why was the model registered as `custom_model`?
3. What is the difference between a raw PyTorch checkpoint and an MLflow model package?
4. Why should model metadata include source-job and dataset information?
5. Why should future models receive new versions?
6. What difference does `/models/` make in `.gitignore`?

Key takeaway

A model asset should be defined as code when possible. Version-controlled YAML preserves both registration configuration and model lineage.

- 6.4 Register the model asset

Objective

Create a managed Version 1 model asset in the Azure ML workspace from the selected checkpoint.

What was implemented

The model was registered using:

```
az ml model create `
  --file infra/azure/models/foliascan-tomato-resnet18.yml `
  --resource-group $RESOURCE_GROUP `
  --workspace-name $WORKSPACE
```

Registered asset:

```
Model:    foliascan-tomato-resnet18
Version:  1
Type:     CUSTOM
```

The source was the existing Azure job output:

```
GPU job
  ↓
training_output
  ↓
best_model.pt
```

The approximately 128 MB checkpoint therefore did not need to be uploaded again from the local machine.

Concepts I should understand

1. Model registration

Registration creates a managed Azure ML object around an existing model artifact.

It provides:

- Model name
- Version
- Metadata
- Traceability
- A stable reference for deployment

2. Registration does not retrain

Registration does **not**:

- Train the model
- Change model weights
- Start GPU compute
- Evaluate the model
- Create an endpoint
- Deploy inference

It manages an already-produced artifact.

3. Direct registration from job output

Because Azure already stores the checkpoint, the model can reference that cloud artifact directly.

Existing Azure artifact



Register reference

rather than:

Download locally



Upload again



Register

4. Tracking versus registration

MLflow tracking stores information about experimental runs.

Model registration marks a particular artifact as a managed version suitable for later lifecycle operations.

Important note

A successful MLflow run does not automatically create a registered Azure ML model asset.

Review questions

1. What does model registration create?

2. Does model registration retrain the model?
3. Why was the model registered directly from the job output?
4. What is the difference between tracking and registration?
5. Why is a registered model useful for deployment?

Key takeaway

Registration turns the selected checkpoint into a managed, named, and versioned Azure ML asset without rerunning training or duplicating the model upload.

- 6.5 Verify the registered version and lineage

Objective

Confirm that the Azure ML model asset has the correct version, type, metadata, and connection to the original GPU training job.

What was implemented

Azure ML showed:

```
Name:      foliascan-tomato-resnet18
Version:    1
Type:       CUSTOM
Created by job:
            cyan_boot_v9vg1214b3
```

The verified lineage was:

```
GPU training job
    ↓
training_output
    ↓
best_model.pt
    ↓
foliascan-tomato-resnet18:1
```

Important metadata included:

```
architecture: resnet18
best-epoch: 10
selection-criterion: lowest-validation-loss
dataset-asset: foliascan-tomato-images:1
framework: pytorch
checkpoint-sha256: ...
```

A local metadata copy was also saved under the ignored artifacts/ directory.

Concepts I should understand

1. Model lineage

Lineage connects the final model asset to the process and inputs that created it.

It helps answer:

Where did this model come from?

2. Registered metadata

Useful metadata should describe:

- Model architecture
- Framework
- Source job
- Source artifact
- Selection criterion
- Dataset version
- Environment version
- Checkpoint fingerprint

3. Post-registration verification

A successful create command only tells us that Azure accepted the registration request.

Verification should confirm that the resulting asset is actually the intended one.

4. Sensitive Azure metadata

Raw CLI metadata may contain internal information such as:

- Subscription IDs
- Workspace resource IDs
- Datastore paths
- Azure resource URIs

Such generated metadata does not need to be committed publicly.

Important note

Registration success is not the same as lineage verification. Always inspect the resulting model asset after creation.

Review questions

1. What does model lineage describe?
2. Why should a registered model link back to its source job?
3. Which metadata is important for reproducibility?
4. Why verify the model after registration?
5. Why should generated Azure metadata stay outside Git?

Key takeaway

A registered model should be traceable from the model asset back to the exact checkpoint and training job that produced it.

- 6.6 Document the model and prepare the deployment handoff

Objective

Document the information required to deploy the registered model correctly in the next phase.

What was implemented

The deployment handoff records:

Registered model:	foliascan-tomato-resnet18:1
Type:	custom_model
Framework:	PyTorch
Architecture:	ResNet18
Classes:	10
Input size:	224 × 224
Checkpoint:	best_model.pt
Source job:	cyan_boot_v9vg1214b3
Source output:	training_output
Selection criterion:	lowest validation loss
Best validation loss:	0.212930
Best validation acc.:	92.8964%

It also includes the environment information and checkpoint fingerprint.

Concepts I should understand

1. Deployment handoff

A deployment handoff is the information needed to turn the registered model into an inference service.

The next phase should know:

- Which model version to use
- How to reconstruct the model
- What image size is expected
- Which class mapping to use
- Which preprocessing is required
- Where the artifact came from

2. Checkpoint as the source of inference metadata

The checkpoint already contains information such as:

```
model_name
model_state_dict
class_to_index
training_config
random_seed
validation_metrics
```

The scoring code can read these instead of duplicating model metadata manually.

3. Registration is not deployment

After Phase 6, several deployment components still do not exist.

The next phase still needs items such as:

- Inference predictor
- Scoring script
- `init()`
- `run()`
- Inference environment
- Endpoint YAML
- Deployment YAML
- Request samples
- Inference tests

4. Training compute versus inference compute

GPU training does not automatically mean GPU inference is necessary.

For FoliaScan's first one-image ResNet18 endpoint, CPU inference was considered sufficient.

Important note

The registered model is **deployment-ready as an asset**, but it is not yet a running API.

Registered model $\neq$ Deployed endpoint

Review questions

1. What information should a deployment handoff contain?
2. Why should inference code reuse metadata from the checkpoint?
3. What remains to be implemented after model registration?
4. Why does GPU training not necessarily require GPU inference?
5. What is the difference between a registered model and a deployed endpoint?

Key takeaway

Phase 6 should finish with a clear deployment contract so the next phase can build the inference service without guessing model architecture, class mapping, preprocessing, or artifact source.

✓ Phase 7 — Deploy an online endpoint

Goal

The goal of Phase 7 is to turn the registered FoliaScan model into a **real-time inference service** that external applications can call through an Azure ML Managed Online Endpoint.

The phase connects the registered model with inference code, a reproducible runtime environment, deployment compute, and a stable API endpoint.

Complete workflow

```
Define the inference request/response contract
↓
Implement and test the inference code
↓
Create a reproducible inference environment
↓
Define endpoint and deployment YAML
↓
Test the deployment locally with Docker
↓
Deploy and invoke the real Azure endpoint
↓
Inspect logs and clean up paid resources
```

Main concepts

1. Inference

Training learns model parameters.

Inference uses the trained model to make predictions on new inputs.

Training:
data → model → update weights

Inference:
new image → trained model → prediction

2. Scoring script

The scoring script connects incoming Azure requests to the model inference logic.

Azure ML expects two important functions:


```
init()
run(raw_data)
```

- `init()` loads the model when the inference container starts.
- `run()` processes each inference request.

3. Endpoint and deployment

These are different objects:

Endpoint

- stable API address
- authentication and traffic routing

Deployment

- model + code + environment + compute

An endpoint can theoretically route traffic to more than one deployment, such as blue, green, or canary deployments.

4. Containerized inference

Azure packages the model, scoring code, dependencies, and runtime into a container.

This makes inference behaviour more consistent between local testing and Azure deployment.

5. Deployment validation

Successful Python unit tests are not enough.

Deployment should also validate:

container startup

- model loading
- health checks
- HTTP request
- real prediction

6. Cost awareness

Managed Online Endpoint compute continues to incur cost while deployed.

Therefore deployment work should include cleanup when the endpoint is no longer needed.

Key takeaway

Phase 7 turns a registered model asset into a callable cloud service. Deployment requires much more than the model file: inference code, runtime dependencies, container behaviour, endpoint configuration, compute, authentication, monitoring, and cleanup must all work together.

- 7.1 Define the inference contract

Objective

Define exactly how a client sends an image to FoliaScan and what response the endpoint returns.

What was implemented

The endpoint supports **single-image, real-time inference**.

Request:

```
{
  "image_base64": "<base64-encoded JPEG or PNG>"
}
```

Response:

```
{
  "predicted_class": "Tomato___Bacterial_spot",
  "predicted_index": 0,
  "confidence": 0.8064,
  "probabilities": {
    "...": 0.0
  }
}
```

The response contains:

- Predicted class name
- Predicted class index
- Confidence
- Probability for each class

Concepts I should understand

1. Inference contract

An inference contract defines the structure that both sides agree on:

```
Client
↓
Expected request format
↓
Endpoint
↓
Expected response format
```

If the request contract changes, clients may also need to change.

2. Base64 encoding

Images are binary data, while JSON is text-based.

Base64 converts image bytes into text so they can be included inside JSON.

```
JPEG/PNG bytes
  ↓
Base64
  ↓
JSON string
```

3. Confidence and probabilities

The model produces probabilities across the ten classes.

The highest-probability class becomes the prediction, while confidence reports that class's probability.

4. Class mapping from the checkpoint

Class names are not hard-coded in the inference service.

They are reconstructed from:

```
class_to_index
```

stored inside the checkpoint.

This keeps training and inference consistent.

Important note

The inference contract should remain stable once an external application starts depending on it. Changing field names or response structure can break clients.

Review questions

1. What is an inference contract?
2. Why is Base64 used for the image request?
3. What is the difference between `predicted_index` and `predicted_class`?
4. What does confidence represent?
5. Why should class names come from the checkpoint rather than be hard-coded?

Key takeaway

Before building an API, define its request and response contract clearly. The model and application need a stable agreement on how data enters and leaves the service.

- 7.2 Implement and test the scoring script locally

Objective

Create reusable inference logic and the Azure ML scoring entry point before creating cloud deployment resources.

What was implemented

Main files:

```
src/foaliascan/inference/  
├── __init__.py  
└── predictor.py
```

```
infra/azure/endpoints/online/code/  
└── score.py
```

Tests were added under:

```
tests/inference/  
├── test_predictor.py  
└── test_score.py
```

The predictor performs:

```
Load best_model.pt  
    ↓  
Validate checkpoint  
    ↓  
Reconstruct ResNet18  
    ↓  
Load weights with strict=True  
    ↓  
Restore class mapping  
    ↓  
Apply evaluation transform  
    ↓  
model.eval()  
    ↓  
torch.inference_mode()  
    ↓  
softmax  
    ↓  
Return prediction
```

Existing functions such as `create_model()` and `create_eval_transform()` were reused rather than duplicated.

Concepts I should understand

1. Predictor versus scoring script

The **predictor** contains reusable ML inference logic.

The **scoring script** adapts that logic to Azure's request lifecycle.

`predictor.py`
→ model-specific inference logic

`score.py`
→ Azure request/response interface

2. `init()`

`init()` runs when the inference container starts.

It:

- Reads the Azure model directory
- Finds `best_model.pt`
- Builds the predictor
- Loads the model once

The model should not be reloaded for every request.

3. `run(raw_data)`

`run()` executes for each request.

It:

Parse JSON
→ decode Base64
→ validate image
→ run inference
→ return response

4. `AZUREML_MODEL_DIR`

Azure provides:

`AZUREML_MODEL_DIR`

to tell the scoring code where the deployed model artifact is located inside the container.

The scoring code should use this environment variable rather than assume a fixed local path.

5. Inference preprocessing consistency

Inference reuses the same evaluation transform used during model validation.

This keeps:

- Image resize
- Crop
- Normalisation

consistent between evaluation and deployment.

6. Input validation

The scoring implementation also handles invalid cases such as:

- Malformed JSON
- Missing image field
- Invalid Base64
- Invalid image bytes
- Invalid checkpoint

The file reports 23 inference-specific tests and 158 full-project tests after this implementation.

Important note

Model loading belongs in `init()`, not `run()`. Reloading a 128 MB model for every request would create unnecessary latency.

Review questions

1. What is the difference between `predictor.py` and `score.py`?
2. When is `init()` executed?
3. When is `run()` executed?
4. What is `AZUREML_MODEL_DIR`?
5. Why should the model be loaded only once?
6. Why should inference reuse the evaluation transform?
7. Why is request validation important?

Key takeaway

Inference logic should be reusable and independently testable. Azure-specific request handling should wrap the ML logic instead of becoming part of the model implementation.

- 7.3 Define the inference environment

Objective

Create a reproducible CPU runtime containing all dependencies required to serve predictions.

What was implemented

Final inference environment:

Name: foliascan-inference-cpu
Version: 2
Device: CPU
Python: 3.11

Important packages include:

```
torch==2.8.0+cpu  
torchvision==0.23.0+cpu  
pillow==11.3.0  
pyyaml==6.0.2  
azureml-inference-server-http==1.5.1
```

Concepts I should understand

1. Inference environment

The inference environment contains the software needed to start the scoring service and execute predictions.

It is separate from the GPU training environment because inference has different requirements.

2. CPU inference

Although GPU compute was useful for training, a single ResNet18 prediction is small enough for CPU inference.

Using CPU:

- Reduces cost
- Simplifies deployment

- Avoids unnecessary GPU quota requirements

3. Runtime dependency

The inference environment must contain not only packages directly imported by `score.py`, but also dependencies imported indirectly.

Example from FoliaScan:

```
predictor.py
  ↓
training.checkpoints
  ↓
training.config
  ↓
yaml
```

Version 1 did not contain PyYAML and failed during container startup.

4. Environment versioning

Instead of silently changing Version 1, the corrected environment became:

```
foliascan-inference-cpu:2
```

This preserves reproducibility.

Important note

A dependency can be required even if it does not appear directly in the scoring script. Transitive dependencies must also be included in the runtime environment.

Review questions

1. Why is the inference environment separate from the training environment?
2. Why was CPU chosen for the first deployment?
3. What is a transitive dependency?
4. Why did the first environment fail?
5. Why was the fix registered as Version 2 rather than replacing Version 1?

Key takeaway

Deployment environments should contain the complete runtime dependency chain. Environment versioning preserves both reproducibility and the history of fixes.

- 7.4 Prepare endpoint and deployment YAML files

Objective

Define the Azure ML endpoint and its deployment as version-controlled infrastructure.

What was implemented

Endpoint:

```
Name: foliascan-tomato-endpoint
Type: Managed online endpoint
Authentication: key
```

Deployment:

```
Name: blue
Model: foliascan-tomato-resnet18:1
Environment: foliascan-inference-cpu:2
```

The deployment also defines:

- Scoring code
- CPU instance type
- Instance count
- Environment variables

Concepts I should understand

1. Managed Online Endpoint

A Managed Online Endpoint provides a stable real-time API managed by Azure ML.

Applications communicate with the endpoint rather than directly with the underlying VM or model container.

2. Endpoint versus deployment

Endpoint

- stable URL
- authentication
- traffic routing

Deployment

- model
- scoring code

→ environment
→ compute

Keeping them separate allows multiple deployments to exist behind one endpoint.

3. Blue deployment

The first deployment was named:

blue

This naming pattern can later support strategies such as:

blue
green
canary

where traffic can be moved between model versions.

4. Version-controlled deployment configuration

Model, environment, compute, and scoring configuration are defined in YAML rather than manually configured only in the portal.

This makes deployment:

- Reviewable
- Repeatable
- Easier to reproduce

5. Container PYTHONPATH

The deployment container mounted the repository under:

/var/azureml-app/foliascan-azure

The original PYTHONPATH was incorrect:

/var/azureml-app/src

The correct value became:

/var/azureml-app/foliascan-azure/src

Without the correct path:

ModuleNotFoundError: No module named 'foliascan'

occurred.

Important note

Local filesystem assumptions do not automatically remain valid inside Docker or Azure containers.

Review questions

1. What is the difference between an endpoint and a deployment?
2. What components belong to the deployment?
3. Why might one endpoint contain several deployments?
4. Why define endpoint infrastructure in YAML?
5. Why did the initial PYTHONPATH fail?
6. What is the purpose of a blue/green deployment pattern?

Key takeaway

The endpoint is the stable interface exposed to clients; the deployment is the implementation behind it. Keeping them separate makes model updates and traffic management easier.

- 7.5 Test the deployment locally with Docker

Objective

Test the complete deployment container locally before creating paid Azure inference compute.

What was implemented

Azure ML local deployment mode created a Docker container similar to the cloud runtime.

Two important problems were detected.

Problem 1 — package import

`ModuleNotFoundError: No module named 'foliascan'`

Cause:

Incorrect PYTHONPATH

Fix:

`/var/azureml-app/foliascan-azure/src`

Problem 2 — missing dependency

`ModuleNotFoundError: No module named 'yaml'`

Cause:

PyYAML missing from inference environment

Fix:

`pyyaml==6.0.2`

and the creation of inference environment Version 2.

After the fixes, the local deployment reached:

Provisioning state: Succeeded

and returned a prediction approximately:

Predicted class: Tomato___Bacterial_spot

Predicted index: 0

Confidence: ~0.8064

The local container and Docker image were removed after validation.

Concepts I should understand

1. Local deployment

Local deployment runs an Azure-like inference container through Docker on the development machine.

Its purpose is to find deployment-specific problems before paying for cloud resources.

2. Unit test versus container test

Unit tests can verify Python logic.

Container testing additionally checks:

- Filesystem layout
- Environment variables
- Runtime dependencies
- Package import paths
- Inference-server startup
- Container health

3. Docker container

The container provides an isolated runtime containing:

Operating environment
Python packages
Scoring code
Model access
Inference server

4. Deployment debugging

A failure may come from:

Model logic
OR
Dependency
OR
Container path

OR
Environment configuration
OR
Inference server

not only from Azure infrastructure.

Important note

A locally working python score.py is weaker evidence than a successfully running deployment container because the local Python process does not reproduce the full container environment.

Review questions

1. Why test with Docker before Azure deployment?
2. What problems did the container test reveal?
3. Why didn't ordinary unit tests detect the PYTHONPATH issue?
4. What caused the missing yaml error?
5. What does successful local provisioning prove?
6. Why was the local Docker deployment deleted afterward?

Key takeaway

Container testing is a strong pre-cloud integration check. It detects runtime and packaging problems that normal Python tests may miss.

- 7.6 Create and invoke the Azure endpoint

Objective

Deploy the registered model to a real Azure ML Managed Online Endpoint and verify remote inference.

What was implemented

The endpoint was created as:

`foliascan-tomato-endpoint`

with:

Provisioning state: Succeeded
Authentication: key
Region: Norway East

The deployment process exposed two important Azure infrastructure issues.

Concepts I should understand

1. Resource provider

The first endpoint creation failed with:

`SubscriptionNotRegistered`

Required Azure resource providers had to be registered.

Examples included:

`Microsoft.MachineLearningServices`
`Microsoft.Compute`
`Microsoft.Network`
`Microsoft.Storage`
`Microsoft.ManagedIdentity`
`Microsoft.ContainerRegistry`
`Microsoft.Insights`
`Microsoft.Quota`

A resource provider must be available in the subscription before Azure can create certain resource types.

2. Managed endpoint compute quota

Training compute quota and online deployment quota are not necessarily the same thing.

The initial deployment used:

Standard_DS3_v2

but encountered a deployment quota constraint.

The final cloud-compatible configuration used:

Standard_DS2_v2

3. Authentication

The endpoint used:

key authentication

A client must provide the endpoint credential when making requests.

Credentials should not be hard-coded or committed to Git.

4. Remote inference

Cloud validation means the request passes through the real Azure service:

```
Client
↓
HTTPS endpoint
↓
Azure authentication
↓
Deployment container
↓
score.py
↓
ResNet18
↓
JSON prediction
```

This validates much more than local model inference.

5. Provisioning state

Succeeded means Azure successfully created the requested resource.

However, endpoint creation alone does not prove inference works; an actual request must also succeed.

Important note

Cloud quota problems may come from a different quota system than the training cluster quotas checked earlier. A working training VM does not guarantee that the same VM family is available for managed online deployment.

Review questions

1. What caused the initial SubscriptionNotRegistered error?
2. What is a resource provider?
3. Why can online deployment quota differ from training compute quota?
4. Why was the endpoint configured with key authentication?
5. What does Provisioning state: Succeeded prove?
6. Why is an actual endpoint invocation still required after provisioning succeeds?

Key takeaway

Real deployment validates Azure infrastructure, authentication, compute quota, container startup, and inference together. A successful local container is necessary but does not replace cloud validation.

- 7.7 Inspect logs, document the endpoint, and delete it

Objective

Verify the final deployment through logs, preserve useful metadata, and remove continuously billed resources.

What was implemented

The deployment reached:

State: Succeeded

Cloud logs showed:

```
SystemSetup: Succeeded
UserContainerImagePull: Succeeded
ModelDownload: Succeeded
UserContainerStart: Succeeded
ContainerReady
POST /score 200
```

These confirm that:

- The environment image was available.
- The registered model was downloaded.
- The scoring container started.
- Health checks passed.
- The inference request succeeded.

Metadata was stored locally under:

```
artifacts/azure/endpoints/
├── foliascan-tomato-endpoint/
│   ├── endpoint-metadata.json
│   ├── deployment-metadata.json
│   └── inference-response.json
```

These files were not intended for Git because they can contain internal Azure metadata.

After successful validation, the Managed Online Endpoint was deleted.

Concepts I should understand

1. Deployment logs

Logs help determine which stage of deployment failed or succeeded.

For example:

```
Image pull
Model download
Container startup
Health probe
Inference request
```

A single Succeeded status does not provide the same diagnostic detail.

2. Health probe

A health probe checks whether the inference container is alive and ready to receive requests.

A container can exist but still be unhealthy.

3. HTTP status

```
POST /score 200
```

means the inference request reached the scoring service and received a successful HTTP response.

4. Deployment metadata

Preserving metadata supports traceability even after the paid endpoint has been removed.

Useful evidence includes:

- Endpoint metadata
- Deployment metadata
- Inference response
- Logs

5. Cleanup

Managed endpoint compute is different from the training clusters configured to scale to zero.

The online endpoint can continue incurring cost while deployed.

Therefore:

Deploy

- verify
- collect evidence
- delete when no longer needed

6. Sensitive metadata

Generated endpoint files may contain:

- Internal Azure IDs
- Resource paths
- Endpoint information

They should not automatically be committed to a public repository.

Important note

Deployment is not complete when the API first returns a prediction. A responsible cloud workflow also includes log inspection, evidence collection, security awareness, and cost cleanup.

Review questions

1. What do deployment logs help diagnose?
2. What is the purpose of a health probe?
3. What does POST /score 200 mean?
4. Why preserve metadata before deleting the endpoint?
5. Why should endpoint metadata stay outside Git?
6. Why was the endpoint deleted after successful validation?
7. Why is endpoint cleanup different from training-cluster scale-to-zero?

Key takeaway

A professional deployment workflow ends with verification and cleanup, not merely with a successful prediction.

✓ Phase 8 — Add a simple application

Goal

The goal of Phase 8 is to build a lightweight user-facing application around the deployed FoliaScan model.

Instead of requiring users to call Azure manually, the application provides a simple interface that accepts an image, sends it to the Azure ML endpoint, and presents the prediction in a readable form.

Complete workflow

```
User uploads image
      ↓
Streamlit UI
      ↓
Reusable endpoint client
      ↓
Base64 + JSON + authentication
      ↓
HTTPS POST
      ↓
Azure ML Managed Online Endpoint
      ↓
ResNet18 prediction
      ↓
JSON response
      ↓
Human-readable result
```

Main concepts

1. Client application

A client application provides a usable interface around the ML service.

The endpoint from Phase 7 already performs inference. Phase 8 adds the layer that makes that service practical for a user.

2. Separation of concerns

The application separates:

```
Presentation logic
      ↓
```


Streamlit UI

Service/integration logic



AzureEndpointClient

The UI should not contain Azure communication code directly.

3. API consumption

The Azure ML endpoint can be consumed like a normal HTTPS API:

image
→ request
→ HTTPS endpoint
→ response

The application does not need the Azure SDK just to make predictions.

4. Configuration and secrets

Runtime configuration is kept outside source code using:

FOLIASCAN_ENDPOINT_URL
FOLIASCAN_ENDPOINT_KEY

Credentials are not committed to the repository.

5. Testing layers

Phase 8 uses different levels of testing:

Unit tests
→ isolated logic

Mocked integration tests
→ client/API behaviour without Azure

Local UI test
→ Streamlit application

Real end-to-end test
→ Streamlit → Azure → model → response

Each level answers a different question.

Key takeaway

Phase 8 connects the deployed ML model to a real user workflow. A usable AI system needs not only a working model and endpoint, but also client architecture, error handling, security, testing, and a clear presentation layer.

- 8.1 Define the client application architecture

Objective

Design a lightweight application that lets a user interact with FoliaScan without using Azure CLI or Python scripts.

What was implemented

The application supports:

- Uploading one tomato-leaf image
- Sending it to the Azure ML endpoint
- Receiving a prediction
- Showing the result in a readable form

The architecture was separated into:

```
app/streamlit_app.py
    ↓
src/foaliascan/client/azure_endpoint.py
    ↓
Azure ML Managed Online Endpoint
```

Concepts I should understand

1. Separation of concerns

Different parts of the system should have different responsibilities.

Streamlit
→ presentation

AzureEndpointClient
→ endpoint communication

The UI does not need to know how authentication, Base64 encoding, HTTP requests, or response validation work.

2. Reusable client

Because the endpoint communication is independent of Streamlit, the same client could later be used by:

- FastAPI

- CLI
- Another Python program
- A different frontend

without rewriting the Azure communication logic.

3. Environment-based configuration

The client reads:

```
FOLIASCAN_ENDPOINT_URL  
FOLIASCAN_ENDPOINT_KEY
```

from environment variables.

This keeps infrastructure-specific configuration and credentials outside application code.

Important note

Do not mix endpoint credentials or HTTP communication directly into the UI code. This makes the application harder to test, reuse, and maintain.

Review questions

1. What is separation of concerns?
2. Why is the endpoint client separate from Streamlit?
3. What advantages does a reusable client provide?
4. Why are endpoint URL and key stored in environment variables?
5. Could the same endpoint client later be reused by another frontend?

Key takeaway

Keep the presentation layer separate from the service-integration layer. This improves testability, maintainability, reuse, and security.

- 8.2 Implement reusable Azure endpoint client

Objective

Implement a small Python client that securely calls the Azure ML Managed Online Endpoint.

What was implemented

The client uses ordinary HTTPS rather than the Azure SDK.

Request flow:

```
JPEG / PNG bytes
    ↓
Base64 encoding
    ↓
JSON payload
    ↓
HTTPS POST
    ↓
Azure ML endpoint
```

Request:

```
{
  "image_base64": "<base64 image>"
}
```

Authentication headers include:

```
Authorization: Bearer <endpoint-key>
Content-Type: application/json
```

The client also implements:

- 30-second timeout
- Configuration validation
- Network error handling
- HTTP error handling
- JSON validation
- Prediction-response validation

Concepts I should understand

1. HTTP API client

The client is responsible for communication between the application and the remote service.

It handles:

serialization
authentication
request
response
errors

2. Timeout

The request uses an explicit timeout:

30 seconds

Without a timeout, an unavailable endpoint could leave the application waiting indefinitely.

3. Response validation

The client expects:

predicted_class
predicted_index
confidence
probabilities

A successful HTTP response does not automatically mean the response contains valid model data.

The structure still needs to be checked.

4. Error translation

The client converts technical failures into understandable application-level errors.

Examples include:

- Missing endpoint configuration
- Empty image
- Network failure
- Timeout
- HTTP 4xx/5xx
- Invalid JSON
- Missing response fields

5. Azure SDK is not required for inference

The endpoint exposes an HTTPS interface.

Therefore:

Application

- ordinary HTTPS
- Azure ML endpoint

is sufficient.

This keeps the client lightweight.

Important note

The endpoint key must never be printed, logged, committed, or sent to the presentation layer.

Review questions

1. Why does the client use HTTPS instead of the Azure SDK?
2. Why is Base64 used in the request?
3. What is the purpose of an HTTP timeout?
4. Why should the JSON response be validated?
5. What types of failures should the client handle?
6. Why should endpoint errors be translated into application-level errors?

Key takeaway

Calling an ML endpoint is more than sending an HTTP request. A reliable client must also manage authentication, serialization, timeouts, network failures, and response validation.

- 8.3 Build the Streamlit user interface

Objective

Create a simple visual interface for uploading a tomato-leaf image and viewing the prediction.

What was implemented

Streamlit was chosen because the main focus of the project is AI engineering, Azure ML, deployment, and MLOps rather than frontend development.

The user flow is:

```
Upload image
↓
Preview
↓
Analyze
↓
Wait for inference
↓
Display prediction
```

The interface includes:

- FoliaScan branding
- JPEG/PNG uploader
- Image preview
- Analyze button
- Loading spinner
- Predicted condition
- Confidence
- Top probabilities
- Educational disclaimer

Concepts I should understand

1. Streamlit

Streamlit provides a fast way to create data/ML applications in Python without building a separate frontend stack.

For FoliaScan, this keeps attention on the ML system rather than unnecessary frontend complexity.

2. Presentation layer

The model returns identifiers such as:

Tomato___Late_blight

The UI displays:

Late Blight

The transformation occurs only in the presentation layer.

The backend API contract remains unchanged.

3. Machine representation versus user representation

Internal representations are designed for programs.

User-facing representations are designed for readability.

Backend:

Tomato___Bacterial_spot

UI:

Bacterial Spot

4. Probability presentation

The UI presents:

- Predicted class
- Confidence
- Top 3 probabilities
- Expandable full probabilities

This provides useful information without making the default interface unnecessarily complex.

Important note

Improving presentation must not change the underlying model output or inference contract.

Review questions

1. Why was Streamlit appropriate for this project?
2. What is the role of the presentation layer?
3. Why are class identifiers converted to human-readable names only in the UI?
4. Why show confidence and top probabilities?
5. Why should presentation changes not modify the backend contract?

Key takeaway

Model output is not automatically good user experience. A presentation layer translates machine-oriented results into clear information for users.

- 8.4 Add tests and local validation

Objective

Validate the client and UI without requiring a running Azure endpoint.

What was implemented

Azure HTTP interactions were mocked.

This made tests:

- Fast
- Deterministic
- Free
- Independent of Azure availability

Presentation tests also checked transformations such as:

```
Tomato___Late_blight
      ↓
Late Blight
```

and probability sorting.

After the application work:

```
Client tests:      20 passed
Full project tests: 178 passed
Ruff:              passed
mypy:              passed
git diff --check:  passed
```

The Streamlit module could also be imported without making an Azure request.

Concepts I should understand

1. Mocking

A mock replaces a real external dependency during testing.

Instead of:

Test

→ real Azure endpoint

the test uses:

Test

→ controlled fake HTTP response

This allows different success and failure scenarios to be tested reliably.

2. Test isolation

Importing:

```
import app.streamlit_app
```

should not automatically:

- Connect to Azure
- Make inference requests
- Consume cloud resources

Importing code and executing external operations should remain separate.

3. Different testing levels

A useful application testing strategy includes:

Unit tests

→ one component

Mocked integration tests

→ components interacting without Azure

Local UI validation

→ real Streamlit interface

Cloud end-to-end test

→ entire production-like path

4. Configuration-error testing

Because the endpoint had been deleted, pressing Analyze without configuration produced:

FOLIASCAN_ENDPOINT_URL is not configured.

This was expected behaviour and verified error handling.

Important note

The routine test suite should not depend on a paid Azure endpoint being available.

Essential command

```
poetry run streamlit run app/streamlit_app.py
```

Review questions

1. What is mocking?
2. Why mock Azure HTTP calls in normal tests?
3. Why should importing the Streamlit module not call Azure?
4. What is the difference between unit, integration, UI, and end-to-end tests?
5. Why is testing configuration errors important?

Key takeaway

Cloud-dependent applications should still have fast and reliable local tests. Real cloud resources should be reserved for the tests that genuinely require them.

- 8.5 Test the app against the real Azure endpoint

Objective

Verify the complete user-to-model workflow using a real Azure ML deployment.

What was implemented

Because the Phase 7 endpoint had already been deleted for cost control, it was temporarily recreated using the existing configuration:

```
Endpoint: foliascan-tomato-endpoint
Deployment: blue
Compute: Standard_DS2_v2
```

The deployment reached:

```
Succeeded
blue: 100% traffic
```

The endpoint URL and key were loaded only into the local PowerShell environment.

A real image was uploaded through Streamlit.

Result:

```
Model prediction:
Tomato___Late_blight
```

```
Displayed:
Late Blight
```

```
Confidence:
85.3%
```

Other probabilities included:

```
Target Spot: 13.4%
Early Blight: 0.7%
```

This verified the full system.

Concepts I should understand

1. End-to-end test

An end-to-end test validates the complete real workflow:

User

- Streamlit
- client
- HTTPS
- authentication
- Azure endpoint
- scoring container
- PyTorch model
- response
- UI

2. Integration boundaries

Many components can work individually while failing when connected.

This test verifies the interfaces between:

- Browser
- Streamlit
- Python client
- Azure authentication
- Endpoint
- Container
- Model
- Response rendering

3. Local configuration

The endpoint URL and key were temporary runtime configuration.

They were not written into source code or Git.

4. Prediction versus ground truth

The displayed Late Blight result is the model's prediction.

It does **not** prove that the plant actually has Late Blight.

This is why the interface includes an educational disclaimer.

Important note

A successful mocked test does not replace a real end-to-end test. Both are useful, but they validate different things.

Review questions

1. What does an end-to-end test validate?
2. Why was the Azure endpoint recreated temporarily?
3. Which components were validated together?
4. Why were endpoint credentials placed only in the local environment?
5. Why does a model prediction not prove the actual disease diagnosis?
6. What is the difference between a mocked integration test and a real end-to-end test?

Key takeaway

The real end-to-end test proved that the complete application path worked from user image upload to Azure model prediction and back to the UI.

- 8.6 Document the application and final workflow

Objective

Turn the working application into a clear and presentable portfolio project through documentation, branding, usability improvements, traceability, and cleanup.

What was implemented

The final UI was polished with:

- FoliaScan logo and page icon
- Concise introduction
- Simple green visual styling
- Controlled image preview size
- Clear prediction area
- Human-readable disease names
- Confidence
- Top 3 predictions
- Expandable probability table
- Educational disclaimer

The design intentionally remained simple rather than becoming a complex dashboard.

Static assets were organised into:

```
assets/  
├── branding/  
├── roadmap/  
└── screenshots/
```

The README was also updated with:

- Project overview
- Architecture
- Model results
- Azure ML workflow
- Streamlit application
- Project phases
- Setup
- Security
- Cost considerations
- Screenshot
- Learning roadmap

A dedicated document was added:

```
docs/client-application.md
```

Concepts I should understand

1. Technical documentation

Documentation should make it possible for another engineer to understand:

- Architecture
- Configuration
- Inference flow
- Error handling
- Local execution
- Cloud validation
- Security
- Cleanup

2. Evidence of validation

The successful real inference screenshot provides evidence that the complete application actually worked.

This is stronger than describing the architecture without demonstrating execution.

3. Secret management

Documentation avoids exposing:

- Endpoint keys
- Subscription IDs
- Endpoint URLs
- Base64 payloads
- Sensitive Azure metadata

4. Cost cleanup

After successful validation:

- The Managed Online Endpoint was deleted.
- Temporary endpoint configuration was removed.

FOLIASCAN_ENDPOINT_URL
FOLIASCAN_ENDPOINT_KEY

Managed endpoint compute continues to incur cost while deployed, so cleanup is part of the engineering workflow.

5. Portfolio-quality engineering

The final system includes more than model accuracy:

```
Working AI system
+
clear architecture
+
usable interface
+
testing
+
documentation
+
security
+
cost awareness
+
evidence of validation
```

Important note

Documentation and branding should explain and present the engineering work; they should not expose credentials or internal Azure information.

Review questions

1. Why is documentation part of an ML engineering project?
2. What should application documentation explain?
3. Why preserve a screenshot of the successful end-to-end test?
4. What Azure information should not appear in public documentation?
5. Why was the endpoint deleted after validation?
6. What makes an ML project portfolio-ready beyond having a working model?

Key takeaway

A complete ML project is not only a trained model or working API. It should also be understandable, usable, testable, secure, documented, and cost-conscious.

✓ Phase 9 — Add GitHub Actions

Goal

The goal of Phase 9 is to automate FoliaScan's important code-quality checks using GitHub Actions so that changes are validated in a clean environment before and after they reach main.

Before CI, these checks depended on the developer remembering to run them manually. After Phase 9:

```
Code change
  ↓
Pull Request
  ↓
GitHub Actions
  ↓
Clean Ubuntu runner
  ↓
Install project
  ↓
pytest + Ruff + mypy + Streamlit smoke test
  ↓
Pass / Fail
```

After the PR is merged:

```
push → main
  ↓
CI runs again
```

Main concepts

1. Continuous Integration — CI

Continuous Integration automatically validates code changes as they are integrated into a shared repository.

CI mainly answers:

Does this change still satisfy the project's quality requirements?

CI does not necessarily deploy the application.

2. GitHub Actions

GitHub Actions is GitHub's automation platform.

A workflow is defined in YAML and describes:

When should automation run?
Where should it run?
What should it do?
What permissions does it need?

3. Workflow, job, step, runner

The basic hierarchy is:

Workflow
↓
Job
↓
Steps

A **runner** is the machine that executes the job.

For FoliaScan:

GitHub-hosted Ubuntu runner
↓
Checkout repository
↓
Install tools and dependencies
↓
Run quality checks
↓
Runner is destroyed

4. Quality gate

A quality gate is a check that must succeed before CI passes.

FoliaScan requires:

pytest passes
AND
Ruff passes
AND
mypy passes
AND
Streamlit imports successfully
↓
CI success

5. CI versus CD

CI
→ validate changes

CD

→ automate release or deployment

FoliaScan implements CI, but intentionally does not automatically train or deploy Azure resources.

6. Security and cost boundary

Routine CI deliberately does not:

- Authenticate to Azure
- Start CPU/GPU compute
- Train models
- Create endpoints
- Invoke paid cloud inference
- Use Azure endpoint keys

This keeps the workflow inexpensive, deterministic, and safer.

Key takeaway

Phase 9 turns important development practices into automated repository rules. Instead of relying on the developer to remember every check, GitHub independently validates each proposed change in a clean environment.

- 9.1 Define the CI strategy

Objective

Decide what the CI pipeline should validate, when it should run, and what should deliberately remain outside automation.

What was implemented

Before CI, important checks were run manually:

```
pytest
Ruff
mypy
Streamlit import check
```

The CI workflow now runs automatically for:

```
Pull request → main
Push → main
```

This gives two levels of validation:

```
Feature branch
  ↓
Pull Request
  ↓
Validate proposed change
  ↓
Merge
  ↓
main
  ↓
Validate final merged state
```

Concepts I should understand

1. CI trigger

A trigger is an event that starts a workflow.

FoliaScan uses:

```
pull_request → main
push → main
```

A normal feature-branch push alone does not trigger this workflow unless that branch has a

pull request targeting main.

2. PR validation

PR validation checks changes **before** they enter the stable branch.

It helps catch problems before merge.

3. Push validation

After merge, CI runs again against main.

This verifies the actual final repository state rather than only the proposed PR state.

4. CI scope

The FoliaScan CI scope is intentionally limited to code-quality validation.

It does not perform Azure training or deployment.

5. CI versus CD

This distinction is important:

Continuous Integration

→ build/test/validate code

Continuous Delivery/Deployment

→ release or deploy software

CI does not have to include cloud deployment.

Important note

Automating every possible operation is not automatically good engineering. Expensive, slow, or security-sensitive operations should only be included when there is a clear reason.

Review questions

1. What is Continuous Integration?
2. What events trigger FoliaScan CI?
3. Why validate both pull requests and pushes to main?
4. Why are Azure training and deployment excluded?
5. What is the difference between CI and CD?

Key takeaway

A good CI strategy defines not only **what to automate**, but also **what not to automate**.

- 9.2 Create the GitHub Actions workflow

Objective

Define the CI pipeline as a version-controlled GitHub Actions workflow.

What was implemented

The workflow is stored at:

```
.github/workflows/ci.yml
```

Its structure is:

```
CI workflow
  ↓
Quality checks job
  ├── Checkout repository
  ├── Install Poetry
  ├── Set up Python
  ├── Install dependencies
  ├── Run pytest
  ├── Run Ruff
  ├── Run mypy
  └── Verify Streamlit import
```

The runner is:

```
ubuntu-latest
```

The job also uses:

```
timeout-minutes: 15
permissions:
  contents: read
```

Concepts I should understand

1. Workflow

A workflow is the complete automation definition stored as YAML.

It includes:

- Triggers
- Jobs
- Permissions

- Runner
- Steps

2. Job

A job is a collection of related steps executed on one runner.

FoliaScan has a quality-check job.

3. Step

A step is one action or shell command.

Examples:

```
Checkout code
Install Poetry
Run pytest
```

If an important step fails, the job fails.

4. GitHub-hosted runner

GitHub creates a temporary machine for the job.

```
New Ubuntu machine
    ↓
Run CI
    ↓
Machine removed
```

It is separate from:

- The local Windows computer
- Azure ML compute
- Azure endpoint VMs

5. Timeout

```
timeout-minutes: 15
```

prevents a broken command from running indefinitely.

6. Least privilege

The workflow needs only:

`contents: read`

This follows the principle of **least privilege**:

Give an automated system only the permissions it actually needs.

Important note

A CI runner is temporary. Do not assume that files, packages, environment variables, or tools from the development computer exist there.

Review questions

1. What is the relationship between a workflow, job, and step?
2. What is a GitHub Actions runner?
3. Why does FoliaScan use ubuntu-latest?
4. Why is a job timeout useful?
5. Why does CI use contents: read?
6. What does least privilege mean?

Key takeaway

A GitHub Actions YAML file defines **when, where, with what permissions, and how** automated validation runs.

- 9.3 Add dependency caching and reproducibility

Objective

Make CI installations reproducible while reducing repeated dependency-download time.

What was implemented

The CI environment explicitly uses:

```
Python 3.11  
Poetry 2.4.1
```

Dependencies are determined by:

```
pyproject.toml  
+  
poetry.lock
```

Poetry caching uses the lock file as the cache dependency:

```
cache: poetry  
cache-dependency-path: poetry.lock
```

Concepts I should understand

1. Clean environment

Each CI run starts from a mostly clean runner.

It cannot rely on packages installed manually on the development machine.

This is useful because it reveals hidden assumptions.

2. Version pinning

CI explicitly specifies the required Python and Poetry versions.

This prevents CI from silently using a different system version.

```
Project Python → 3.11  
CI Python      → 3.11
```

3. Dependency locking

`pyproject.toml` describes dependency requirements.

`poetry.lock` records the resolved dependency versions.

Using the lock file makes different installations more consistent.

4. Dependency cache

Without caching:

CI run 1 → download packages

CI run 2 → download the same packages again

With caching:

`poetry.lock` unchanged

↓

reuse cached downloads

If dependency state changes, a new cache can be created.

5. Cache versus installation

Caching does **not** replace:

`poetry install`

The installation step still runs.

The cache only avoids unnecessary repeated downloads and setup work.

6. CI reproducibility

Reproducibility depends on more than source code.

Important elements include:

Source code

+

Python version
+
Poetry version
+
poetry.lock
+
installation process

Important note

A cache is primarily a performance optimization. The lock file is what provides dependency reproducibility.

Review questions

1. Why does CI start from a clean environment?
2. Why should Python and Poetry versions be controlled?
3. What role does poetry.lock play?
4. What problem does dependency caching solve?
5. Does caching replace poetry install?
6. What is the difference between dependency locking and dependency caching?

Key takeaway

Locking provides reproducibility; caching provides speed. They solve different problems.

- 9.4 Run automated quality checks

Objective

Run the important local quality checks automatically for every CI validation.

What was implemented

The CI workflow runs four main quality gates.

Full test suite

```
poetry run pytest
```

At the end of the project:

178 tests

covered areas such as:

- Dataset processing
- Training
- Evaluation
- MLflow
- Cloud helpers
- Inference
- Endpoint client
- Streamlit presentation

Ruff

```
poetry run ruff check .
```

Ruff checks issues such as:

- Unused imports
- Import ordering
- Style problems
- Selected Python code-quality rules

mypy

```
poetry run mypy
```

mypy performs static type checking.

Streamlit import smoke test

CI also checks that:

```
import app.streamlit_app
```

works successfully.

Importing the UI must not:

- Call Azure
- Require endpoint credentials
- Generate cloud usage

Concepts I should understand

1. Automated quality gate

A quality rule becomes enforceable when CI automatically checks it.

Instead of:

Developers should remember to run tests.

the repository effectively says:

Tests fail

↓

CI fails

↓

Problem is visible on the PR

2. Different checks catch different problems

pytest → behaviour

Ruff → code quality

mypy → type correctness

Smoke test → component starts/imports successfully

One does not replace another.

3. CI smoke test

The Streamlit smoke test is intentionally lightweight.

It does not perform full browser or cloud inference testing.

It simply verifies that the application module can start at a basic level without external cloud dependencies.

4. Non-zero exit code

CLI tools communicate failure through their process exit code.

If an important command returns a non-zero code:

```
Step fails
  ↓
Job fails
  ↓
Workflow fails
```

Important note

CI quality checks should be fast enough to run frequently. Expensive full Azure training is not necessary to verify ordinary Python changes.

Review questions

1. What does pytest validate?
2. What different problems do Ruff and mypy detect?
3. What does the Streamlit import smoke test prove?
4. Why should importing the app not contact Azure?
5. What causes a GitHub Actions job to fail?
6. What is a quality gate?

Key takeaway

CI turns development conventions into automated rules that are visible directly on every pull request.

- 9.5 Validate the workflow on a real Pull Request

Objective

Confirm that the workflow works on GitHub's real infrastructure rather than only appearing correct locally.

What was implemented

The CI work was submitted through a real pull request.

GitHub automatically started:

CI / Quality checks

Result:

All checks passed

The first real workflow run took approximately:

3 minutes

After merge, the push → main trigger ran the workflow again.

Later pull requests also automatically triggered CI, demonstrating that it became part of the normal development process.

Concepts I should understand

1. Local validation versus CI validation

Before the real PR test, we knew:

YAML can be parsed
Commands work locally

After GitHub CI succeeded, we additionally knew:

GitHub detects the workflow
Triggers work
Ubuntu runner works
Python setup works
Poetry setup works
Dependencies install cleanly

Tests work on Linux
Ruff works
mypy works
Streamlit imports

2. Clean-runner validation

A clean CI environment can expose hidden dependencies on:

- Local files
- Machine-specific configuration
- Uncommitted files
- Installed packages not declared in the project
- Operating-system assumptions

3. Cross-platform validation

Local development occurred on Windows while GitHub Actions runs on Linux.

Successful CI therefore provides additional confidence that the code is not unnecessarily tied to the development machine.

4. CI as part of development workflow

CI is most useful when it continuously validates future changes rather than being executed once as a demonstration.

Important note

Works on my machine is weaker evidence than:

Works from a clean checkout
with declared dependencies
on an independent runner

Review questions

1. Why was a real pull request necessary to validate CI?
2. What does a clean runner detect that local tests may miss?
3. Why is Linux CI useful when development occurs on Windows?
4. What did the push-to-main run verify?
5. Why should CI continue running on future PRs?

Key takeaway

A workflow is not truly validated until it succeeds on the actual automation platform and a clean runner.

- 9.6 Add CI status/documentation and finalize the project

Objective

Document the CI workflow, make its status visible, and clearly define its security and cost boundaries.

What was implemented

A GitHub Actions status badge was added to the README:

CI | `passing`

The README documents:

- Workflow triggers
- Python version
- Poetry version
- pytest
- Ruff
- mypy
- Streamlit smoke test
- Dependency caching
- Azure exclusions

A dedicated file was also added:

`docs/ci.md`

It explains:

- CI purpose
- Triggers
- Runner
- Dependency installation
- Caching
- Quality gates
- Security boundaries
- Cost considerations

Concepts I should understand

1. CI status badge

A status badge provides a quick visual indication of the most recent workflow status.

It shows that automated validation exists, but it should not be interpreted as proof that every part of the cloud system is currently deployed or healthy.

2. CI documentation

A CI pipeline should explain:

What is automated?
What is not automated?
When does it run?
What does failure mean?
Why were these boundaries selected?

3. Secrets management

FoliaScan CI does not require:

- Azure credentials
- Subscription identifiers
- Endpoint keys
- Deployment permissions

Therefore those secrets are intentionally not added to GitHub Actions.

The safest unnecessary secret is one that is never provided at all.

4. Cost boundary

Azure operations remain outside routine CI.

This prevents a normal pull request from accidentally triggering:

GPU training
Managed endpoint deployment
Paid Azure compute

5. CI badge versus system health

A green CI badge means:

Current repository validation passed

It does not mean:

Azure endpoint is currently deployed
Model is currently serving traffic
Cloud infrastructure has zero issues

Important note

Avoid giving automation cloud permissions merely because the platform supports secrets. Permissions and credentials should only be introduced when the workflow genuinely requires them.

Review questions

1. What does a CI status badge communicate?
2. What should CI documentation explain?
3. Why does FoliaScan CI not contain Azure secrets?
4. How does excluding cloud deployment reduce cost risk?
5. Why is a green CI badge not the same as production-system health?
6. Why should security boundaries be documented explicitly?

Key takeaway

A professional CI pipeline should be understandable as well as functional. Its documentation should clearly show what is automated, what remains manual, and why those boundaries exist.